

Physics-Informed Extreme Learning Machines for Uncertainty Quantification in Computational Mechanics

A Bachelor Thesis

Sagar Suman Karna 222953
Department of Mechanical Engineering
Chair for Reliability Engineering

Supervisor: Prof. Dr.-Ing. Matthias Faes

Advisor: Dr. Marcos Valdebenito

April 13, 2026

Declaration

I hereby declare that the work presented in this thesis is original and has been carried out by me under academic guidance. This thesis has not been submitted elsewhere for the award of any academic degree. All sources of information used have been properly cited.

April 13, 2026

Sagar Karna

Acknowledgement

I am especially thankful to my advisor, Dr. Marcos Valdebenito, for his insightful discussions, constructive feedback, and constant support during the development of this thesis. I am also thankful to the faculty members of the Department of Reliability Engineering for providing the academic foundation necessary to complete this research.

Abstract

Many problems in computational mechanics are governed by differential equations whose analytical solutions are available only for simplified configurations. In practical applications, variations in material properties, geometric parameters, or loading conditions often require repeated numerical solutions, which can become computationally expensive, particularly in uncertainty quantification studies.

The work presented here examines whether Physics-Informed Extreme Learning Machines (PIELMs) can serve as efficient surrogates for mechanical systems. The method combines the ELM architecture with physical constraints enforced at collocation points. This reduces learning to the solution of a deterministic linear system, avoiding iterative training and nonlinear optimization.

The proposed approach is first validated using the Euler–Bernoulli cantilever beam subjected to a uniformly distributed load. The Physics-Informed Extreme Learning Machine accurately reproduces the analytical solution for the deterministic case and is subsequently extended to account for uncertainty in Young’s modulus and the second moment of area by treating these parameters as additional inputs. This enables efficient evaluation of beam deflection and estimation of statistical quantities such as mean response, variance, and probability distributions.

To further demonstrate the flexibility of the framework, the transverse vibration of a stretched string governed by the one-dimensional wave equation is considered as a time-dependent test case. This example illustrates that the proposed method can handle both static and dynamic problems within a unified physics-informed formulation.

The results show that Physics-Informed Extreme Learning Machines provide a computationally efficient and physically consistent alternative to conventional numerical solvers and traditional physics-informed neural networks, particularly for uncertainty quantification in mechanical systems.

Contents

List of Abbreviations	9
List of Figures	10
List of Tables	12
1 Introduction	13
2 Literature Review	16
2.1 Uncertainty Quantification Methods	16
2.2 Surrogate Modeling and Reduced-Order Methods	17
2.3 Neural Networks and Physics-Informed Learning	18
2.4 Extreme Learning Machines	18
2.4.1 Strengths and Limitations of ELM	19
2.5 Research Gap and Positioning	19
3 Extreme Learning Machine	20
3.1 Regression Problem Statement	20
3.2 Architecture of the Extreme Learning Machine	21
3.3 Hidden-Layer Mapping	22
3.4 Output Representation	22
3.5 Matrix Formulation and Training	22
3.6 Strengths and Limitations of the ELM Framework	23
3.7 Discussion	24

4	Physics-Informed Extreme Learning Machine	25
4.1	Extension of ELM to PDE-Constrained Learning	25
4.2	Collocation Enforcement of Governing Equations	26
4.3	Formulation of Residual Systems	26
4.4	Imposition of Boundary Conditions	27
4.5	Constrained Linear and KKT Formulation	28
4.6	Regularization and Numerical Solution of the KKT System	28
4.6.1	Role of Tikhonov Regularization	28
4.6.2	Regularized KKT System	29
4.6.3	Choice of the Regularization Parameter	29
4.6.4	Logarithmic Parameter Search	30
4.6.5	Diagnostic-Based Selection Criteria	30
4.6.6	Numerical Properties of the KKT System	30
4.7	Discussion	31
4.8	Completion of the Deterministic Methodological Foundation	31
5	PIELM for Uncertainty Quantification	32
5.1	Conceptual Motivation	32
5.2	Parametric Formulation with Uncertainty	33
5.3	Augmented Space Formulation	34
5.4	Collocation Point Generation in the Presence of Uncertainty	35
5.4.1	Interior Collocation Points	35
5.4.2	Boundary Collocation Points	36
5.4.3	Dimensional Scaling	36
5.5	Sampling Strategies	37
5.5.1	Sampling Methods Considered	37
5.5.2	Coverage, Scalability, and Surrogate Accuracy Impact	40
5.6	Activation Function Selection	41
5.6.1	Smoothness Requirements	41

5.6.2	Required Types of Activation Functions	42
5.6.3	Impact on Surrogate Accuracy	42
5.6.4	Adopted Activation Function	43
5.7	Implementation Aspects	43
5.8	Role in Uncertainty Propagation	44
5.8.1	Surrogate-Based Monte Carlo Evaluation	44
5.8.2	Estimation of Moments	44
5.8.3	Estimator Reliability	45
5.8.4	Empirical Cumulative Distribution Function	45
5.8.5	Comparison with Classical Approaches	46
5.9	Summary	46
6	Example 1 — Cantilever Beam	47
6.1	Problem Setup	47
6.2	Deterministic Study	48
6.2.1	Problem Setup	48
6.2.2	Results	49
6.2.3	Discussion	49
6.3	Activation Function Study	50
6.3.1	Problem Setup	50
6.3.2	Results	50
6.3.3	Discussion	51
6.4	Hyperparameter Convergence Study	52
6.4.1	Neuron Convergence	52
6.4.2	Collocation Point Convergence	52
6.5	Reproducibility Across Random Initializations	53
6.6	Sampling Study	54
6.6.1	Problem Setup	54
6.6.2	Results	55

6.6.3	Discussion	56
6.7	Uncertainty Quantification	57
6.7.1	Mean and Variance	57
6.7.2	Empirical Cumulative Distribution Function	58
6.7.3	Sensitivity Analysis	60
6.7.4	Discussion	61
6.8	Summary	62
7	Example 2 — Vibrating String	64
7.1	Problem Setup	64
7.2	Deterministic PIELM Validation	65
7.2.1	Problem Setup	65
7.2.2	Results	66
7.2.3	Discussion	67
7.3	Hyperparameter Convergence Study	67
7.3.1	Neuron Convergence	67
7.3.2	Collocation Point Convergence	68
7.4	Reproducibility Across Random Initializations	68
7.5	Uncertainty in Wave Speed	69
7.5.1	Mean and Standard Deviation	70
7.5.2	Empirical Cumulative Distribution Function	71
7.5.3	Sensitivity Analysis	72
7.6	Discussion	73
7.7	Summary	74
8	Conclusions and Future Work	75
8.1	Conclusions	75
8.1.1	Key Findings	75
8.1.2	Main Contributions	76

8.2	Limitations	76
8.3	Future Work	77
8.4	Final Remarks	77

List of Abbreviations

ELM	Extreme Learning Machine
PIELM	Physics-Informed Extreme Learning Machine
PINN	Physics-Informed Neural Network
PDE	Partial Differential Equation
UQ	Uncertainty Quantification
KKT	Karush–Kuhn–Tucker
MC	Monte Carlo
LHS	Latin Hypercube Sampling
CDF	Cumulative Distribution Function
CoV	Coefficient of Variation
ECDF	Empirical Cumulative Distribution Function
FEM	Finite Element Method
RMSE	Root Mean Square Error

List of Figures

3.1	Schematic illustration of an Extreme Learning Machine.	21
5.1	Illustration of full factorial (tensor-product) sampling in a two-dimensional augmented space ($d_x = 1, d_\theta = 1$).	38
5.2	Illustration of Monte Carlo sampling in a two-dimensional augmented space. The collocation points are distributed randomly over the domain $\Omega \times \Theta$	39
5.3	Illustration of Latin Hypercube Sampling in a two-dimensional augmented space. Each dimension is partitioned into equal bins, ensuring marginal coverage.	40
5.4	Qualitative comparison of smooth activation functions considered for the PIELM framework. The Gaussian activation is infinitely differentiable and localized, properties that are advantageous in physics-informed learning. .	42
6.1	Comparison between the deterministic PIELM prediction using a sigmoid activation function and the analytical solution for the cantilever beam problem.	49
6.2	Comparison of beam deflection obtained using different activation functions.	51
6.3	Comparison of beam deflection obtained using different sampling strategies in the augmented domain. The reference analytical solution is shown for $E = 1.0, I = 1.0$	55
6.4	Mean deflection with $\pm 2\sigma$ uncertainty band obtained using PIELM and the analytical solution. The analytical band is computed by propagating the uniform distributions of E and I through the closed-form solution. .	58
6.5	Empirical cumulative distribution function of beam deflection at $x = 2.00$.	59

7.1	Comparison between the analytical solution and the deterministic PIELM prediction for the vibrating string at $t = 0.3$ and $c = 1.0$	66
7.2	Mean displacement and $\pm 2\sigma$ uncertainty band for the vibrating string with uncertain wave speed $c \sim \mathcal{U}[0.9, 1.1]$ at $t = 0.3$	70
7.3	Empirical cumulative distribution function of the displacement at the probe point $x = 0.5, t = 0.3$	71

List of Tables

6.1	Accuracy and computational time for different activation functions. . . .	51
6.2	Convergence of relative L^2 error with respect to number of hidden neurons L_h (left, $N_r = 100$) and number of collocation points N_r (right, $L_h = 40$). . . .	53
6.3	Statistics of PIELM performance over 30 random initializations.	53
6.4	Comparison of selected cumulative distribution function values at $x = 2.00$	59
6.5	Comparison of PIELM-based sensitivities with analytical values at $x = L/2$	60
7.1	Convergence of relative L^2 error with respect to number of hidden neurons L_{neurons} for the deterministic vibrating string problem.	67
7.2	Convergence of relative L^2 error with respect to number of interior collocation points N_r for the deterministic vibrating string problem.	68
7.3	Statistics of PIELM performance for the vibrating string problem over 30 random initializations.	68
7.4	Comparison of selected cumulative distribution function values at the probe point $x = 0.5, t = 0.3$	72
7.5	Sensitivity of displacement with respect to wave speed at $x = 0.5, t = 0.3, c = 1.0$	73

Chapter 1

Introduction

Differential equations play a central role in engineering and scientific modeling, as they describe how physical quantities such as displacement, stress, and temperature evolve under applied loads and boundary conditions. For a limited class of idealized problems, closed-form analytical solutions can be derived. However, most practical systems are too complex for such solutions, and numerical approximation methods are therefore required.

Classical approaches, including methods for finite difference and finite element ,discretize the physical domain and transform the governing equations into systems of algebraic equations [1]. These techniques are well established and highly accurate, but their computational cost becomes significant when repeated evaluations of the model are required.

Such repeated evaluations arise in parametric studies, design optimization, and uncertainty quantification (UQ) [2]. In these settings, material properties, loading conditions, or geometric parameters are treated as uncertain variables, and the deterministic problem must be solved for many different parameter realizations. Sampling-based approaches, such as Monte Carlo methods, therefore require a large number of simulations [3]. When each simulation is computationally expensive, the overall cost quickly becomes prohibitive.

Surrogate and reduced-order modeling techniques provide an effective way to alleviate this limitation by approximating the response of the system at significantly lower computational cost [4]. In parallel, machine learning methods—particularly neural networks—have emerged as powerful tools for function approximation. Neural networks can learn complex nonlinear relationships directly from the data and, in principle, approximate any continuous function given sufficient capacity and training data [5]. However, purely data-driven models often require large datasets, careful hyperparameter tuning, and may produce predictions that do not strictly satisfy known physical laws.

Physics-informed machine learning offers an alternative approach in which the governing

equations and boundary conditions are incorporated directly into the learning process. Instead of solving the system repeatedly for each parameter realization, a single model is trained to represent the solution in a combined physical and parametric domain [6, 7]. By embedding physical constraints into the model formulation, this approach improves consistency, robustness, and generalization, particularly in data-limited settings.

The Extreme Learning Machine (ELM) is the specific method considered in this thesis. An ELM is a single-hidden-layer neural network in which hidden-layer weights and biases are randomly assigned and remain fixed, while only output-layer weights are determined during training [8]. This structure reduces the learning process to the solution of a linear system, enabling fast and deterministic training with reduced sensitivity to hyperparameter choices.

By enforcing the residual of the governing differential equation and the boundary conditions at collocation points, a Physics-Informed Extreme Learning Machine (PIELM) is obtained [9]. Rather than approximating individual solution instances, the PIELM learns the underlying solution behavior over a combined spatial and parametric domain, making it particularly suitable for repeated-query settings, such as uncertainty analysis.

Two benchmark problems are used to evaluate the proposed approach: the Euler–Bernoulli cantilever beam and the vibrating string governed by the wave equation. These problems represent two distinct classes of physical systems. The beam problem corresponds to a static boundary value problem in structural mechanics [10], while the vibrating string represents a dynamic system that involves spatial and temporal evolution [11, 12]. The latter introduces an additional independent variable, increasing the dimensionality and complexity of the problem while remaining analytically tractable.

For the beam problem, uncertainty is introduced in the material and geometric parameters, specifically Young’s modulus E and the second moment of the area I . By treating these quantities as additional inputs, the PIELM is extended to approximate the solution over a combined spatial and parametric domain. A similar approach is adopted for the vibrating string, where uncertainty may be introduced in parameters such as wave speed or tension.

This unified formulation enables efficient prediction of system responses across a wide range of parameter realizations without repeated numerical simulations. As a result, statistical quantities such as the mean and variance can be computed efficiently [2]. Furthermore, the probabilistic behavior of the system is analyzed using sampling techniques, including Monte Carlo and Latin Hypercube sampling [13].

Overall, this work demonstrates that the PIELM framework provides a computationally efficient and physically consistent approach for solving deterministic and stochastic differential

equations within a unified and computationally efficient modeling framework.

Chapter 2

Literature Review

Physics-informed surrogate modeling for uncertainty quantification builds upon several established fields. This chapter reviews key developments in uncertainty propagation, surrogate modeling, neural network approximation, and physics-informed learning. It concludes by identifying the research gap addressed in this thesis.

2.1 Uncertainty Quantification Methods

Uncertainty quantification (UQ) aims to assess the impact of uncertain input parameters on the response of mathematical models. In engineering applications, material properties, geometric characteristics, and loading conditions are often not known with complete certainty. As a result, deterministic simulations must be performed for multiple parameter realizations to estimate statistical properties of the system response.

Monte Carlo (MC) simulation is one of the most widely used approaches for uncertainty propagation due to its simplicity and general applicability [14]. In this framework, uncertain parameters are sampled from prescribed probability distributions, and the deterministic model is evaluated repeatedly. Statistical quantities such as the mean, variance, and probability density functions are then estimated from the resulting ensemble of responses.

Despite its simplicity, MC simulation exhibits slow convergence, with the statistical error decreasing at a rate proportional to $N^{-1/2}$, where N is the number of samples. Consequently, a large number of evaluations may be required to obtain accurate results. For high-fidelity models, this leads to significant computational cost. This challenge becomes more severe as the number of uncertain parameters increases, since the computational cost of each individual model evaluation typically grows with problem complexity. A related but distinct challenge is the curse of dimensionality, which affects structured

sampling and surrogate construction methods, where the number of required evaluations grows rapidly with the dimension of the parameter space [15].

Structured sampling techniques can improve efficiency. Latin Hypercube Sampling (LHS), for example, enforces stratification across each input dimension and provides improved space-filling properties compared to purely random sampling [13]. Nevertheless, these approaches do not eliminate the fundamental challenge of repeatedly evaluating the underlying deterministic model.

When governing equations are solved numerically using finite difference or finite element methods [1], this repeated evaluation becomes particularly expensive. This computational burden motivates the development of surrogate models capable of approximating system responses efficiently.

2.2 Surrogate Modeling and Reduced-Order Methods

Surrogate modeling replaces computationally expensive models with approximations that are significantly cheaper to evaluate. These approaches are especially valuable in uncertainty quantification, where repeated simulations are required.

Early surrogate methods include response surface techniques, which construct polynomial or interpolatory approximations from sampled input-output data. While effective for low-dimensional problems, their performance often deteriorates in the presence of strong nonlinearities or high-dimensional parameter spaces.

Projection-based reduced-order models provide an alternative by constructing low-dimensional subspaces that capture dominant features of the solution space [4]. These methods can achieve high accuracy; however, they are often intrusive, requiring modifications to the governing equations or numerical solvers.

Nonintrusive surrogate models treat the original solver as a black box and rely solely on observed input-output data. Machine learning approaches naturally fall into this category, offering flexibility without requiring changes to the underlying physical model.

Overall, surrogate and reduced-order methods reduce computational cost, but involve trade-offs between accuracy, generality, and implementation complexity. Projection-based approaches often provide high fidelity but lack flexibility, whereas nonintrusive methods are more versatile but may struggle in strongly nonlinear or high-dimensional regimes [4].

2.3 Neural Networks and Physics-Informed Learning

Neural networks are widely used as universal function approximators. The universal approximation theorem states that even a single-hidden-layer network can approximate continuous functions on compact domains to a desired level of accuracy [5]. This makes neural networks attractive for surrogate modeling of complex systems.

However, purely data-driven neural networks do not inherently enforce physical laws. Their predictions may violate conservation principles or boundary conditions, particularly outside the training domain.

Physics-informed machine learning offers an alternative approach by embedding governing equations directly into the training process. Instead of relying solely on data, the residuals of differential equations are incorporated into the loss function, ensuring that the learned model respects the underlying physics [6, 7].

Physics-Informed Neural Networks (PINNs) are a prominent realization of this idea. In PINNs, training involves solving a non-convex optimization problem that balances data fitting and equation residual minimization. While this approach provides strong physical consistency, it typically requires careful tuning of hyperparameters and may suffer from training instability.

Earlier work by Lagaris [16] introduced neural network trial functions constructed to satisfy boundary conditions analytically. This concept has influenced modern physics-informed approaches based on residual enforcement at collocation points.

2.4 Extreme Learning Machines

Extreme Learning Machines (ELMs) provide a computationally efficient alternative to conventional neural network training. They employ a simplified training paradigm for single-hidden-layer neural networks, in which the hidden-layer parameters are randomly initialized and not updated during training, while the output weights are computed directly by solving a linear system rather than through iterative optimization [8].

Unlike traditional neural networks, which rely on iterative gradient-based optimization, ELMs reduce training to solving a linear least-squares problem. This results in fast and deterministic computation.

The deterministic nature of ELM training improves reproducibility and reduces sensitivity to hyperparameter selection. However, standard ELMs do not incorporate physical constraints and therefore cannot directly enforce governing differential equations.

2.4.1 Strengths and Limitations of ELM

ELMs offer several advantages. Training is fast, deterministic, and does not require iterative optimization, which simplifies implementation and reduces computational effort [8].

However, ELMs also have limitations. The shallow architecture may limit expressive capability compared to deep neural networks, particularly for highly complex or high-dimensional problems [8]. In addition, scalability may become a concern when dealing with large datasets or fine-resolution solution fields [8].

2.5 Research Gap and Positioning

The literature demonstrates substantial progress in uncertainty quantification, surrogate modeling, neural networks, and physics-informed learning. However, several challenges remain.

Classical UQ methods rely on repeated deterministic solutions of partial differential equations, resulting in significant computational cost. Projection-based reduced-order models are efficient but typically intrusive. Physics-Informed Neural Networks require solving non-convex optimization problems and may suffer from training instability. Standard Extreme Learning Machines, while computationally efficient, do not incorporate physical constraints.

The integration of Extreme Learning Machines with physics-based residual enforcement to construct operator surrogates for uncertainty propagation remains relatively underexplored in the existing literature, particularly in the context of uncertainty propagation. In particular, the development of models capable of learning solution operators over combined physical and parametric domains has received limited attention.

This thesis addresses this gap by developing a Physics-Informed Extreme Learning Machine framework for efficient and physically consistent uncertainty propagation through operator surrogate modeling over combined physical and parametric domains.

Chapter 3

Extreme Learning Machine

Extreme Learning Machines (ELMs) are single-hidden-layer neural networks whose hidden weights and biases are randomly assigned and kept fixed. Training therefore reduces to solving a linear least-squares problem for the output weights. The following sections formalize this architecture as a foundation for the physics-informed extension in Chapter 4.

3.1 Regression Problem Statement

Consider a supervised regression problem defined by a set of N input–output pairs

$$\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N, \quad (3.1)$$

where all vectors are assumed to be column vectors unless stated otherwise. The inputs and outputs satisfy

$$\mathbf{x}^{(i)} \in \mathbb{R}^d, \quad (3.2)$$

$$\mathbf{y}^{(i)} \in \mathbb{R}^m, \quad (3.3)$$

and N denotes the number of training samples.

The objective is to approximate an unknown mapping

$$\mathbf{y} = \mathcal{F}(\mathbf{x}), \quad (3.4)$$

using an Extreme Learning Machine. For later matrix formulations, the output data are

assembled into the matrix

$$\mathbf{Y} = \begin{bmatrix} \mathbf{y}^{(1)\top} \\ \vdots \\ \mathbf{y}^{(N)\top} \end{bmatrix} \in \mathbb{R}^{N \times m}. \quad (3.5)$$

3.2 Architecture of the Extreme Learning Machine

An Extreme Learning Machine is a single-hidden-layer feedforward neural network consisting of:

- an input layer of dimension d ,
- a hidden layer with L neurons,
- an output layer of dimension m .

The defining characteristic of the ELM is that the parameters of the hidden layer are randomly generated and kept fixed, while only the parameters of the output layer are learned during training [8]. This distinguishes ELMs from conventional neural networks, which typically rely on iterative gradient-based optimization.

Figure 3.1 illustrates a representative ELM architecture.

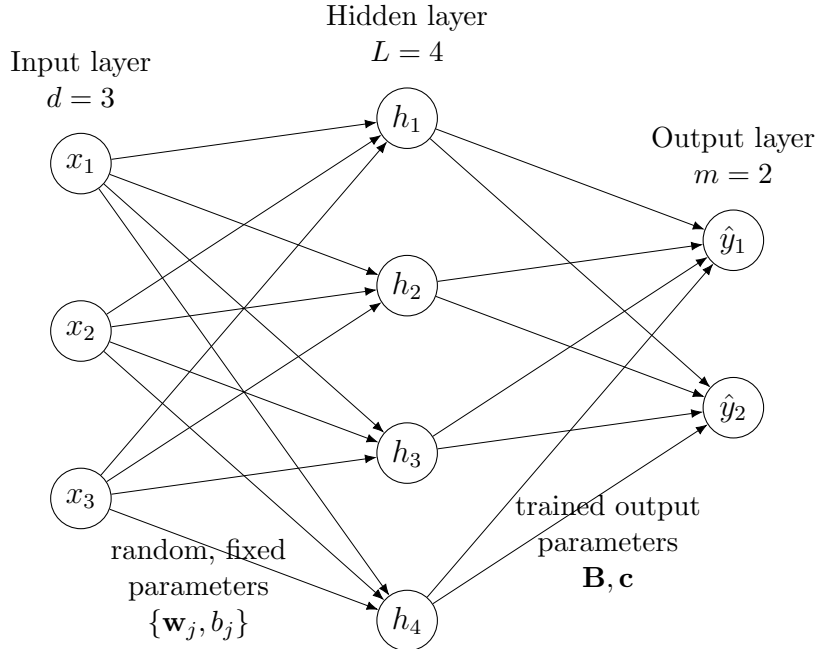


Figure 3.1: Schematic illustration of an Extreme Learning Machine.

3.3 Hidden-Layer Mapping

Each hidden neuron performs an affine transformation followed by a nonlinear activation function. Let

- $\mathbf{w}_j \in \mathbb{R}^d$ denote the weight vector,
- $b_j \in \mathbb{R}$ denote the bias,
- $g(\cdot)$ denote the activation function.

The output of the j -th hidden neuron is

$$h_j(\mathbf{x}) = g(\mathbf{w}_j^\top \mathbf{x} + b_j). \quad (3.6)$$

Collecting all hidden neuron outputs:

$$\mathbf{h}(\mathbf{x}) \in \mathbb{R}^L. \quad (3.7)$$

The parameters $\{\mathbf{w}_j, b_j\}$ are randomly assigned and remain fixed.

3.4 Output Representation

The model output is expressed as

$$\hat{\mathbf{y}}(\mathbf{x}) = \mathbf{B}^\top \mathbf{h}(\mathbf{x}) + \mathbf{c}, \quad (3.8)$$

where $\mathbf{B}$ and $\mathbf{c}$ are learned parameters.

3.5 Matrix Formulation and Training

For all samples, the model output can be written as

$$\hat{\mathbf{Y}} = \mathbf{H}\mathbf{B} + \mathbf{1}\mathbf{c}^\top, \quad (3.9)$$

where $\mathbf{H} \in \mathbb{R}^{N \times L}$ is the hidden layer output matrix, $\mathbf{B} \in \mathbb{R}^{L \times M}$ is the output weight matrix, $\mathbf{c} \in \mathbb{R}^M$ is the bias vector, and $\mathbf{1} \in \mathbb{R}^{N \times 1}$ is a vector of ones.

Defining the augmented matrices

$$\tilde{\mathbf{H}} = [\mathbf{H} \ \mathbf{1}], \quad \tilde{\mathbf{B}} = \begin{bmatrix} \mathbf{B} \\ \mathbf{c}^\top \end{bmatrix}, \quad (3.10)$$

the model can be written compactly as

$$\hat{\mathbf{Y}} = \tilde{\mathbf{H}}\tilde{\mathbf{B}}. \quad (3.11)$$

Training then reduces to the least-squares problem

$$\tilde{\mathbf{B}} = \arg \min_{\tilde{\mathbf{B}}} \left\| \tilde{\mathbf{H}}\tilde{\mathbf{B}} - \mathbf{Y} \right\|_F^2, \quad (3.12)$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

The solution is given by

$$\tilde{\mathbf{B}} = \tilde{\mathbf{H}}^\dagger \mathbf{Y}, \quad (3.13)$$

where $\dagger$ denotes the Moore–Penrose pseudoinverse [8, 17].

3.6 Strengths and Limitations of the ELM Framework

The Extreme Learning Machine offers several advantages that make it attractive for regression and surrogate modeling tasks:

- Training is deterministic and based on a linear least-squares formulation.
- No iterative gradient-based optimization is required.
- Training is computationally efficient and fast.
- The method exhibits reduced sensitivity to hyperparameter tuning.

At the same time, certain limitations should be acknowledged:

- The shallow architecture limits expressivity compared to deep neural networks.
- A large number of hidden neurons may be required for complex problems.
- Performance depends on the random initialization of hidden-layer parameters.
- Standard ELMs do not enforce governing physical constraints.

- Scalability may become challenging for very large models.

These aspects highlight the trade-off between computational efficiency and representational flexibility.

3.7 Discussion

The Extreme Learning Machine provides a computationally efficient alternative to traditional neural network training. By avoiding iterative optimization and relying on a linear system solution, it offers significant advantages in terms of speed and robustness. These properties are particularly relevant in applications such as uncertainty quantification, where models must be evaluated repeatedly across a range of parameters.

However, the limitations of the ELM framework must also be considered. Its shallow structure restricts its ability to capture highly complex patterns compared to deep architectures, and the absence of embedded physical constraints limits its direct applicability to scientific problems governed by differential equations.

The efficiency of ELMs is attractive, but physical consistency is non-negotiable for engineering systems. The next chapter combines both by extending the ELM framework with physics-based constraints.

Chapter 4

Physics-Informed Extreme Learning Machine

This chapter extends the Extreme Learning Machine (ELM) introduced in Chapter 3 to PDE-constrained learning. The objective is to construct a neural approximation of a physical field that satisfies governing differential equations and boundary conditions. The resulting formulation follows the general philosophy of physics-informed learning, while preserving the linear training structure characteristic of ELMs [6–9].

In contrast to standard Physics-Informed Neural Networks (PINNs), which typically rely on nonlinear optimization of composite loss functions, the present formulation yields a deterministic constrained linear system for the unknown output parameters [6, 7, 9].

4.1 Extension of ELM to PDE-Constrained Learning

Let $u(\mathbf{x})$ denote a scalar physical field defined on a domain $\Omega \subset \mathbb{R}^d$.

Using the ELM formulation from Chapter 3, the approximation is written as

$$\hat{u}(\mathbf{x}) = \mathbf{b}^\top \mathbf{h}(\mathbf{x}) + c, \quad (4.1)$$

where

$$\mathbf{b} \in \mathbb{R}^L, \quad (4.2)$$

$$c \in \mathbb{R}, \quad (4.3)$$

$$\mathbf{h}(\mathbf{x}) \in \mathbb{R}^L. \quad (4.4)$$

The hidden-layer parameters remain randomly generated and fixed, while only the

output parameters $\mathbf{b}$ and c are unknown, as in the standard ELM framework [8].

To make the model physics-informed, this approximation must satisfy a governing differential equation together with the associated boundary conditions.

4.2 Collocation Enforcement of Governing Equations

Assume that the physical field satisfies

$$\mathcal{L}[u(\mathbf{x})] = f(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad (4.5)$$

where $\mathcal{L}[\cdot]$ is a linear differential operator and $f(\mathbf{x})$ is a prescribed source term.

The equation is enforced at a finite set of interior collocation points

$$\{\mathbf{x}_r^{(i)}\}_{i=1}^{N_r} \subset \Omega. \quad (4.6)$$

Rather than discretizing the domain in the classical finite element or finite difference sense, the PIELM approach enforces the governing equation directly at selected collocation points, in the spirit of physics-informed residual enforcement [6, 9, 16].

4.3 Formulation of Residual Systems

The residual of the governing equation at an interior collocation point is defined as

$$R(\mathbf{x}_r^{(i)}) = \mathcal{L}[\hat{u}(\mathbf{x}_r^{(i)})] - f(\mathbf{x}_r^{(i)}). \quad (4.7)$$

Applying the differential operator to the ELM approximation gives

$$\mathcal{L}[\hat{u}(\mathbf{x})] = \mathbf{b}^\top \mathcal{L}[\mathbf{h}(\mathbf{x})] + c \mathcal{L}[1]. \quad (4.8)$$

Enforcing $R(\mathbf{x}_r^{(i)}) = 0$ for all interior collocation points yields the system

$$\mathbf{H}_r \mathbf{b} + c \mathbf{l}_r = \mathbf{f}_r, \quad (4.9)$$

where $\mathbf{H}_r \in \mathbb{R}^{N_r \times L}$ is the residual matrix defined by

$$\mathbf{H}_r = \begin{bmatrix} \mathcal{L}[h_1(\mathbf{x}_r^{(1)})] & \cdots & \mathcal{L}[h_L(\mathbf{x}_r^{(1)})] \\ \vdots & \ddots & \vdots \\ \mathcal{L}[h_1(\mathbf{x}_r^{(N_r)})] & \cdots & \mathcal{L}[h_L(\mathbf{x}_r^{(N_r)})] \end{bmatrix}, \quad (4.10)$$

and $\mathbf{l}_r \in \mathbb{R}^{N_r}$ contains the values $\mathcal{L}[1]$ evaluated at the collocation points, while $\mathbf{f}_r \in \mathbb{R}^{N_r}$ collects the source term values $f(\mathbf{x}_r^{(i)})$.

This shows that PDE enforcement leads to a linear system in the unknown output parameters.

4.4 Imposition of Boundary Conditions

Let the boundary conditions be given in operator form by

$$\mathcal{B}[u(\mathbf{x})] = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega, \quad (4.11)$$

where $\mathcal{B}[\cdot]$ denotes a boundary operator.

These conditions are enforced at a set of boundary collocation points

$$\{\mathbf{x}_b^{(j)}\}_{j=1}^{N_b} \subset \partial\Omega. \quad (4.12)$$

Applying the boundary operator to the approximation gives

$$\mathcal{B}[\hat{u}(\mathbf{x})] = \mathbf{b}^\top \mathcal{B}[\mathbf{h}(\mathbf{x})] + c \mathcal{B}[1]. \quad (4.13)$$

Enforcing the boundary conditions at all boundary points yields

$$\mathbf{H}_b \mathbf{b} + c \mathbf{l}_b = \mathbf{g}_b, \quad (4.14)$$

where $\mathbf{H}_b \in \mathbb{R}^{N_b \times L}$ is the boundary matrix, $\mathbf{l}_b \in \mathbb{R}^{N_b}$ collects the values of $\mathcal{B}[1]$, and $\mathbf{g}_b \in \mathbb{R}^{N_b}$ contains the prescribed boundary data.

4.5 Constrained Linear and KKT Formulation

Combining the interior residual equations and the boundary conditions leads to the constrained least-squares problem

$$\min_{\tilde{\mathbf{b}}} \frac{1}{2} \left\| \tilde{\mathbf{H}}_r \tilde{\mathbf{b}} - \mathbf{f}_r \right\|_2^2, \quad (4.15)$$

$$\text{subject to } \tilde{\mathbf{H}}_b \tilde{\mathbf{b}} = \mathbf{g}_b, \quad (4.16)$$

where the augmented unknown vector is defined as

$$\tilde{\mathbf{b}} = \begin{bmatrix} \mathbf{b} \\ c \end{bmatrix} \in \mathbb{R}^{L+1}. \quad (4.17)$$

The augmented matrices are

$$\tilde{\mathbf{H}}_r = \begin{bmatrix} \mathbf{H}_r & \mathbf{1}_r \end{bmatrix}, \quad \tilde{\mathbf{H}}_b = \begin{bmatrix} \mathbf{H}_b & \mathbf{1}_b \end{bmatrix}. \quad (4.18)$$

Introducing the Lagrangian

$$\mathcal{J}(\tilde{\mathbf{b}}, \boldsymbol{\lambda}) = \frac{1}{2} \left\| \tilde{\mathbf{H}}_r \tilde{\mathbf{b}} - \mathbf{f}_r \right\|_2^2 + \boldsymbol{\lambda}^\top (\tilde{\mathbf{H}}_b \tilde{\mathbf{b}} - \mathbf{g}_b), \quad (4.19)$$

where $\boldsymbol{\lambda}$ is the vector of Lagrange multipliers. The first-order optimality conditions then yield the Karush–Kuhn–Tucker (KKT) system

$$\begin{bmatrix} \tilde{\mathbf{H}}_r^\top \tilde{\mathbf{H}}_r & \tilde{\mathbf{H}}_b^\top \\ \tilde{\mathbf{H}}_b & \mathbf{0} \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{b}} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \tilde{\mathbf{H}}_r^\top \mathbf{f}_r \\ \mathbf{g}_b \end{bmatrix}. \quad (4.20)$$

This saddle-point system enforces the boundary conditions exactly, while satisfying the governing differential equation in a least-squares sense [9].

4.6 Regularization and Numerical Solution of the KKT System

4.6.1 Role of Tikhonov Regularization

In practice, the augmented interior normal matrix $\tilde{\mathbf{H}}_r^\top \tilde{\mathbf{H}}_r$ may be ill-conditioned, particularly when the hidden-layer features are strongly correlated or when the collocation set is limited. A standard stabilization mechanism for such least-squares systems is Tikhonov

regularization [17].

To preserve the interpretation of c as a bias parameter, regularization is applied only to the neuron-weight block. Define

$$\mathbf{R} = \begin{bmatrix} \mathbf{I}_L & \mathbf{0} \\ \mathbf{0}^\top & 0 \end{bmatrix}. \quad (4.21)$$

The regularized primal block is then written as

$$\tilde{\mathbf{H}}_r^\top \tilde{\mathbf{H}}_r \longrightarrow \tilde{\mathbf{H}}_r^\top \tilde{\mathbf{H}}_r + \alpha \mathbf{R}, \quad \alpha > 0. \quad (4.22)$$

Regularization is therefore not introduced to alter the physical model or the exact form of the boundary constraints. Its role is to stabilize the interior least-squares problem by penalizing ill-conditioned directions in the feature subspace.

4.6.2 Regularized KKT System

With Tikhonov regularization, the saddle-point system becomes

$$\begin{bmatrix} \tilde{\mathbf{H}}_r^\top \tilde{\mathbf{H}}_r + \alpha \mathbf{R} & \tilde{\mathbf{H}}_b^\top \\ \tilde{\mathbf{H}}_b & \mathbf{0} \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{b}} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \tilde{\mathbf{H}}_r^\top \mathbf{f}_r \\ \mathbf{g}_b \end{bmatrix}. \quad (4.23)$$

In exact arithmetic, the boundary constraints are enforced exactly [9]. In floating-point arithmetic, they are satisfied up to the tolerance of the linear solver and the conditioning of the saddle-point matrix [17].

4.6.3 Choice of the Regularization Parameter

The effectiveness of Tikhonov regularization depends on the choice of α [17]. Since $\mathbf{H}_r^\top \mathbf{H}_r$ has no intrinsic, problem-independent scale, α should be selected relative to the magnitude of the interior feature matrix. The scaling is based on $\mathbf{H}_r$ rather than $\tilde{\mathbf{H}}_r$ so that the bias term does not influence the regularization level. A simple scale-aware choice is

$$\alpha = \varepsilon \frac{\text{trace}(\mathbf{H}_r^\top \mathbf{H}_r)}{L}, \quad (4.24)$$

where L is the number of hidden neurons and $\varepsilon > 0$ is a dimensionless tuning parameter.

4.6.4 Logarithmic Parameter Search

In practice, the appropriate value of ε may vary over several orders of magnitude. A pragmatic strategy is therefore to perform a logarithmic search over a range such as

$$\varepsilon \in \{10^{-10}, 10^{-9}, \dots, 10^{-4}\}. \quad (4.25)$$

For each candidate value, the corresponding regularized KKT system is assembled and solved.

4.6.5 Diagnostic-Based Selection Criteria

For each solution, the following diagnostic quantities may be monitored:

$$\|\mathbf{H}_r \mathbf{b} + c \mathbf{l}_r - \mathbf{f}_r\|_2, \quad \|\tilde{\mathbf{b}}\|_2, \quad \|\boldsymbol{\lambda}\|_2, \quad \|\mathbf{H}_b \mathbf{b} + c \mathbf{l}_b - \mathbf{g}_b\|_2. \quad (4.26)$$

A practical selection strategy is:

1. retain only values of ε for which the boundary residual remains at solver tolerance;
2. among those, prefer values that minimize the interior residual;
3. if several values perform similarly, select the one producing more moderate norms of $\tilde{\mathbf{b}}$ and $\boldsymbol{\lambda}$.

This makes the regularization trade-off explicit: very small values may leave the system numerically unstable, whereas excessively large values may over-penalize the solution and degrade residual fitting [17].

4.6.6 Numerical Properties of the KKT System

Even after regularization, the KKT matrix remains symmetric indefinite. Although the regularized primal block is positive definite on the regularized subspace for $\alpha > 0$, the zero block associated with the Lagrange multipliers prevents the full system from being symmetric positive definite. As a result, methods designed for SPD systems, such as Cholesky factorization or conjugate gradients, are not appropriate here. Instead, the system should be solved using methods for symmetric indefinite linear systems, such as pivoted $LDL^\top$ factorizations [18].

4.7 Discussion

Physics-informed learning need not come at the cost of computational simplicity. In the present formulation, PDE residuals and boundary conditions are enforced at collocation points, while the unknown output parameters are obtained from a deterministic constrained linear system. Compared with standard PINNs, which are typically trained through nonlinear optimization, this yields a simpler and more reproducible training procedure [6, 7, 9].

This structure is especially attractive in repeated-query settings, such as surrogate evaluation and uncertainty quantification, where repeated nonlinear retraining would be undesirable. The ultimate accuracy of the method, however, still depends on the collocation strategy, the hidden feature map, and the numerical conditioning of the resulting linear system.

4.8 Completion of the Deterministic Methodological Foundation

The PIELM formulation reduces PDE-constrained learning to a deterministic linear-algebra problem. Unlike standard PINNs, it does not require the tuning of learning rates or iterative loss-balancing parameters during training [6, 9]. The following chapters build on this deterministic foundation for the treatment of specific physical problems and, subsequently, uncertainty quantification.

Chapter 5

PIELM for Uncertainty Quantification

This chapter extends the deterministic Physics-Informed Extreme Learning Machine (PIELM) framework developed in Chapter 4 to problems involving parametric uncertainty.

Up to this point, the formulation has been restricted to deterministic partial differential equations. However, in practical engineering applications, material properties, geometric parameters, and loading conditions are rarely known with complete certainty. As a result, the solution of the governing equations depends on uncertain parameters, and the problem shifts naturally from deterministic analysis to uncertainty propagation.

Importantly, this extension does not require a fundamental change in the underlying learning algorithm. Instead, uncertainty is incorporated through an augmentation of the input space, allowing the same deterministic training framework to be applied in a broader setting.

5.1 Conceptual Motivation

Let a physical field $u(\mathbf{x})$ satisfy a governing equation in a spatial domain Ω . In the deterministic setting, the solution can be written as

$$u = u(\mathbf{x}). \quad (5.1)$$

If the governing equation depends on uncertain parameters $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^{d_\theta}$, the solution becomes a parametric mapping

$$u = u(\mathbf{x}, \boldsymbol{\theta}). \quad (5.2)$$

A direct approach would solve the governing PDE repeatedly for many realizations of $\boldsymbol{\theta}$. As discussed in earlier chapters, repeated deterministic solution of the PDE can become computationally prohibitive in uncertainty quantification and related many-query settings [14].

The central idea in this work is therefore to treat the uncertain parameters as additional inputs to the neural approximation. Rather than solving many deterministic problems, a single surrogate model is constructed over the combined spatial–parametric domain.

This reformulates the problem as the approximation of a *stochastic solution manifold*. Equivalently, the PIELM can be interpreted as learning a *solution operator* that maps spatial coordinates and uncertain parameters directly to the physical response. Once trained, this operator enables rapid evaluation for new parameter realizations without requiring further numerical solves.

5.2 Parametric Formulation with Uncertainty

In many engineering and physical applications, uncertainty enters the governing equations through external forcing, boundary data, and intrinsic model parameters. These parameters may represent material properties, geometric characteristics, constitutive coefficients, or effective quantities arising from model reduction or homogenization. Such parameters are typically not known exactly and are therefore described probabilistically.

Let the vector $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^{d_\theta}$ collect all uncertain model parameters. The uncertainty in $\boldsymbol{\theta}$ is modeled by treating it as a random vector with a prescribed joint probability density function p_θ ,

$$p_\theta : \Theta \rightarrow \mathbb{R}_+, \quad (5.3)$$

which is assumed to be known a priori, either from experimental observations, expert knowledge, or upstream uncertainty quantification analyses [14].

The presence of parametric uncertainty fundamentally alters the nature of the solution. Instead of a deterministic field defined solely over the physical domain Ω , the solution becomes a parametric map

$$u : \Omega \times \Theta \rightarrow \mathbb{R}, \quad (5.4)$$

which assigns to each spatial location $\mathbf{x}$ and each parameter realization $\boldsymbol{\theta}$ a corresponding system response. For fixed $\boldsymbol{\theta}$, the function $u(\cdot, \boldsymbol{\theta})$ represents the solution of a deterministic boundary value problem, whereas the full map encodes the dependence of the solution on uncertain parameters.

The governing equations are written in operator form as a parametric partial differential

equation

$$\mathcal{L}_{\mathbf{x}}[u(\mathbf{x}, \boldsymbol{\theta}); \boldsymbol{\theta}] = f(\mathbf{x}, \boldsymbol{\theta}), \quad \mathbf{x} \in \Omega, \quad (5.5)$$

where the differential operator $\mathcal{L}_{\mathbf{x}}[\cdot; \boldsymbol{\theta}]$ acts on the spatial variable but depends parametrically on $\boldsymbol{\theta}$. This dependence may be explicit, for instance through coefficients multiplying spatial derivatives, or implicit through constitutive relations.

Boundary conditions are treated analogously and are written as

$$\mathcal{B}[u(\mathbf{x}, \boldsymbol{\theta}); \boldsymbol{\theta}] = g(\mathbf{x}, \boldsymbol{\theta}), \quad \mathbf{x} \in \partial\Omega, \quad (5.6)$$

where both the boundary operator $\mathcal{B}[\cdot; \boldsymbol{\theta}]$ and the prescribed boundary data $g(\mathbf{x}, \boldsymbol{\theta})$ may depend on the uncertain parameters.

From a conceptual perspective, the parametric problem defines a family of deterministic boundary value problems indexed by $\boldsymbol{\theta}$. From a computational standpoint, however, solving this family by repeated deterministic simulations rapidly becomes infeasible as the dimension of the parameter space increases [14]. This observation motivates the construction of a single surrogate model that approximates the full parametric solution map $u(\mathbf{x}, \boldsymbol{\theta})$ over the joint space $\Omega \times \Theta$.

5.3 Augmented Space Formulation

Let

$$\Omega \subset \mathbb{R}^{d_x}, \quad \Theta \subset \mathbb{R}^{d_\theta}. \quad (5.7)$$

The augmented domain is defined as

$$\mathcal{Z} = \Omega \times \Theta. \quad (5.8)$$

A generic point in this domain is written as

$$\mathbf{z} = (\mathbf{x}, \boldsymbol{\theta}). \quad (5.9)$$

The augmented input vector may equivalently be written as

$$\mathbf{z} = \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\theta} \end{bmatrix} \in \mathbb{R}^{d_x + d_\theta}. \quad (5.10)$$

Using this notation, the parametric PIELM approximation is written as

$$\hat{u}(\mathbf{x}, \boldsymbol{\theta}) = \hat{u}(\mathbf{z}) = \mathbf{b}^\top \mathbf{h}(\mathbf{z}) + c. \quad (5.11)$$

In this formulation, the dependence on the uncertain parameters is captured entirely through the augmented input vector $\mathbf{z}$. It is important to emphasize, however, that the governing equations remain purely differential with respect to the spatial variable $\mathbf{x}$. The uncertain parameters enter the problem only in a parametric manner, for instance through coefficients, source terms, or boundary data, and no derivatives with respect to $\boldsymbol{\theta}$ appear in the differential operators.

This allows a single trained model to represent the system response across the full admissible range of parameter variability. From a computational perspective, the formulation replaces repeated deterministic simulations by a single global surrogate defined over the augmented space.

5.4 Collocation Point Generation in the Presence of Uncertainty

Define the augmented domain

$$\mathcal{Z} = \Omega \times \Theta. \quad (5.12)$$

Two distinct collocation sets are introduced: interior collocation points and boundary collocation points.

5.4.1 Interior Collocation Points

Interior points are sampled from the augmented domain:

$$\mathcal{Z}_r = \left\{ \mathbf{z}_r^{(i)} = (\mathbf{x}_r^{(i)}, \boldsymbol{\theta}_r^{(i)}) \right\}_{i=1}^{N_r} \subset \Omega \times \Theta. \quad (5.13)$$

In the parametric setting, each interior collocation point must simultaneously resolve the physical coordinates and the uncertain inputs. Therefore, the augmented collocation points live in the product space $\mathcal{Z} = \Omega \times \Theta$ and have total dimension $d_x + d_\theta$. This construction ensures that the learned surrogate enforces the governing operator not only across the physical domain, but also across the full range of admissible parameter realizations.

The residual of the governing equation is enforced at each augmented interior collocation point:

$$\mathcal{L}_{\mathbf{x}}[\hat{u}(\mathbf{x}_r^{(i)}, \boldsymbol{\theta}_r^{(i)}); \boldsymbol{\theta}_r^{(i)}] = f(\mathbf{x}_r^{(i)}, \boldsymbol{\theta}_r^{(i)}). \quad (5.14)$$

5.4.2 Boundary Collocation Points

Boundary collocation points are defined as

$$\mathcal{Z}_b = \left\{ \mathbf{z}_b^{(j)} = (\mathbf{x}_b^{(j)}, \boldsymbol{\theta}_b^{(j)}) \right\}_{j=1}^{N_b} \subset \partial\Omega \times \Theta. \quad (5.15)$$

Here, the physical boundary locations $\mathbf{x}_b^{(j)}$ are selected on the boundary $\partial\Omega$, for example on each boundary segment or at a prescribed set of boundary points determined by the geometry, while the uncertain parameters $\boldsymbol{\theta}_b^{(j)}$ are sampled to span the full parameter space Θ .

In this way, the boundary conditions are enforced over all relevant realizations of the uncertain inputs. For each boundary location, the parameter space is systematically sampled so that the surrogate remains consistent with the boundary conditions across Θ .

Accordingly, the boundary operator is enforced at each augmented boundary collocation point as

$$\mathcal{B}[\hat{u}(\mathbf{x}_b^{(j)}, \boldsymbol{\theta}_b^{(j)}); \boldsymbol{\theta}_b^{(j)}] = g(\mathbf{x}_b^{(j)}, \boldsymbol{\theta}_b^{(j)}), \quad j = 1, \dots, N_b. \quad (5.16)$$

5.4.3 Dimensional Scaling

If d_x denotes the spatial dimension and d_θ the number of uncertain parameters, then the effective input dimension becomes

$$d_{\text{total}} = d_x + d_\theta. \quad (5.17)$$

As this total dimension increases, the number of collocation points required for accurate coverage of the augmented domain generally increases as well. This reflects the curse of dimensionality, whereby the number of required samples grows rapidly with the dimension of the input space [15]. This makes the sampling strategy a central component of the method.

5.5 Sampling Strategies

The distribution of collocation points in the augmented domain $\mathcal{Z} = \Omega \times \Theta$ plays a central role in the numerical stability, approximation accuracy, and computational cost of the PIELM framework [14]. Sampling determines how effectively the governing equations and boundary conditions are enforced throughout the physical–parametric space.

It is useful to distinguish between two roles of sampling in this thesis. First, sampling is used to generate interior and boundary collocation points during surrogate construction. Second, sampling is employed during uncertainty propagation to estimate statistical quantities such as the mean, variance, and empirical cumulative distribution function. The present section focuses on sampling strategies for surrogate construction.

5.5.1 Sampling Methods Considered

Three sampling strategies are considered in this work:

- Full factorial (tensor-product) sampling,
- Monte Carlo sampling,
- Latin Hypercube Sampling (LHS).

These strategies differ in how they balance coverage of the augmented domain against scalability as the total dimension increases.

Full Factorial Design

In a full factorial design, each coordinate of the augmented input space $\mathcal{Z} = \Omega \times \Theta$ is discretized independently, and the full set of collocation points is obtained through the Cartesian product of the one-dimensional discretizations. Let the augmented variable be denoted by $\mathbf{z} = (z_1, \dots, z_{d_x+d_\theta})$, where d_x and d_θ denote the spatial and parametric dimensions, respectively. If the k -th coordinate is discretized by the finite set $\mathcal{Z}_k = \{z_k^{(1)}, \dots, z_k^{(n_k)}\}$, then the resulting interior collocation set is

$$\mathcal{Z}_r = \mathcal{Z}_1 \times \mathcal{Z}_2 \times \dots \times \mathcal{Z}_{d_x+d_\theta}. \quad (5.18)$$

The total number of interior points is therefore

$$N_r = |\mathcal{Z}_r| = \prod_{k=1}^{d_x+d_\theta} |\mathcal{Z}_k|. \quad (5.19)$$

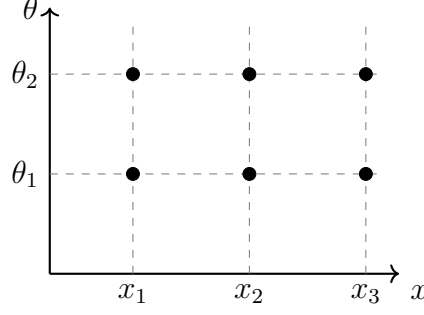


Figure 5.1: Illustration of full factorial (tensor-product) sampling in a two-dimensional augmented space ($d_x = 1$, $d_\theta = 1$).

If each coordinate uses the same number n of points, this simplifies to

$$N_r = n^{d_x + d_\theta}. \quad (5.20)$$

Full factorial sampling provides structured and deterministic coverage of the augmented domain. It resolves interactions between spatial coordinates and uncertain parameters exhaustively, which can lead to high surrogate accuracy in low-dimensional settings. However, the exponential growth of N_r with $d_x + d_\theta$ leads to the curse of dimensionality [2,14], making this approach impractical as the number of uncertain parameters increases.

As illustrated in Figure 5.1, full factorial sampling in a two-dimensional augmented space is obtained by taking all possible combinations of the discretized spatial locations $\{x_1, x_2, x_3\}$ and parameter values $\{\theta_1, \theta_2\}$. Each point therefore corresponds to one element of the Cartesian product $\Omega \times \Theta$.

Monte Carlo Sampling

Monte Carlo (MC) sampling generates collocation points randomly and independently according to prescribed probability distributions [14]. In contrast to tensor-product designs, Monte Carlo sampling does not require explicit gridding of each coordinate direction and therefore scales more naturally in higher-dimensional augmented spaces.

The interior collocation set is written as

$$\mathcal{Z}_r = \left\{ \mathbf{z}_r^{(i)} = (\mathbf{x}_r^{(i)}, \boldsymbol{\theta}_r^{(i)}) \right\}_{i=1}^{N_r} \subset \Omega \times \Theta. \quad (5.21)$$

In the absence of additional prior information, a natural default choice is to sample the physical coordinates uniformly over Ω and the uncertain parameters from their prescribed joint probability distribution. This rule explores the physical domain without bias while remaining consistent with the uncertainty model.

As illustrated in Figure 5.2, Monte Carlo sampling generates an unstructured set of collocation points in the augmented space. This avoids the tensor-product growth of full factorial designs, but the resulting samples may exhibit clustering and uneven coverage.

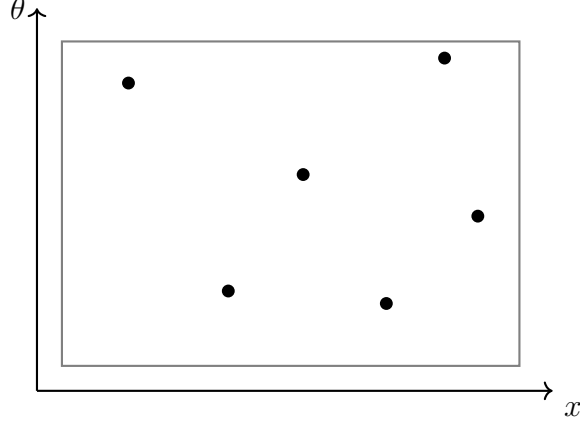


Figure 5.2: Illustration of Monte Carlo sampling in a two-dimensional augmented space. The collocation points are distributed randomly over the domain $\Omega \times \Theta$.

Latin Hypercube Sampling

Latin Hypercube Sampling (LHS) improves upon standard Monte Carlo sampling by enforcing stratification in each input dimension [13, 14]. The construction is typically carried out in a normalized unit hypercube and the resulting samples are then mapped to the physical and parametric domains.

Let $d = d_x + d_\theta$ denote the total dimension of the augmented variable $\mathbf{z} = (\mathbf{x}, \boldsymbol{\theta})$. LHS first generates samples

$$\mathbf{u}^{(i)} \in [0, 1]^d, \quad i = 1, \dots, N_r, \quad (5.22)$$

such that each coordinate direction is partitioned into N_r equal bins and exactly one sample is drawn from each bin in each coordinate.

The samples are then mapped to the augmented space. A common component-wise transformation is

$$\mathbf{z}^{(i)} = T(\mathbf{u}^{(i)}) = (T_x(u_1^{(i)}, \dots, u_{d_x}^{(i)}), T_\theta(u_{d_x+1}^{(i)}, \dots, u_d^{(i)})). \quad (5.23)$$

If the physical coordinates are sampled uniformly over a rectangular bounding box $\Omega = \prod_{\ell=1}^{d_x} [x_{\ell,\min}, x_{\ell,\max}]$, an affine map can be used:

$$x_\ell^{(i)} = x_{\ell,\min} + u_\ell^{(i)}(x_{\ell,\max} - x_{\ell,\min}), \quad \ell = 1, \dots, d_x. \quad (5.24)$$

For uncertain parameters with prescribed marginal distributions, the inverse-CDF transform is often used:

$$\theta_m^{(i)} = F_{\theta_m}^{-1}\left(u_{d_x+m}^{(i)}\right), \quad m = 1, \dots, d_\theta. \quad (5.25)$$

In this way, LHS is carried out in the unit hypercube while the resulting collocation points lie in the augmented domain

$$\mathcal{Z}_r = \left\{ \mathbf{z}_r^{(i)} = (\mathbf{x}_r^{(i)}, \boldsymbol{\theta}_r^{(i)}) \right\}_{i=1}^{N_r} \subset \Omega \times \Theta. \quad (5.26)$$

LHS guarantees uniform marginal coverage in each coordinate direction while avoiding the exponential growth associated with full factorial designs. In many moderate-dimensional problems, this leads to better space-filling behavior than purely random Monte Carlo sampling for a fixed sample budget [13, 14].

As illustrated in Figure 5.3, each axis is divided into equal bins and the final two-dimensional sample set is constructed so that each bin is occupied exactly once in each coordinate direction. This provides stratification in each marginal dimension without forming a full tensor-product grid.

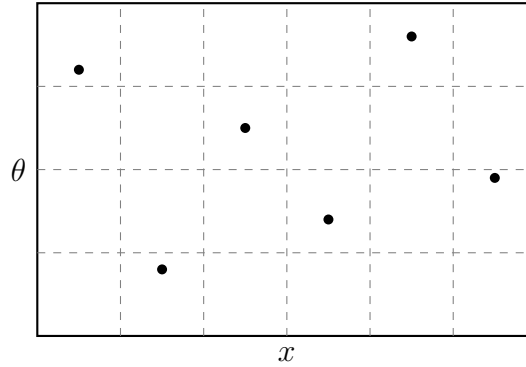


Figure 5.3: Illustration of Latin Hypercube Sampling in a two-dimensional augmented space. Each dimension is partitioned into equal bins, ensuring marginal coverage.

5.5.2 Coverage, Scalability, and Surrogate Accuracy Impact

The practical choice of sampling strategy is governed by a trade-off between:

- **Coverage:** how uniformly the augmented domain is explored,
- **Scalability:** how the required number of points grows with $d_x + d_\theta$,

- **Surrogate accuracy:** how effectively the collocation set supports stable and accurate enforcement of PDE and boundary constraints.

Full factorial sampling provides excellent coverage and can deliver high surrogate accuracy in low-dimensional settings, but it quickly becomes computationally prohibitive because of the curse of dimensionality [2, 14]. Monte Carlo sampling scales more easily with dimension, but may require many points to avoid gaps and clustering. LHS often offers a useful compromise by improving coverage without incurring the full cost of tensor-product discretization.

Sampling affects both the numerical conditioning of the PIELM system and the accuracy of the surrogate across the augmented domain.

5.6 Activation Function Selection

The choice of activation function plays an important role in the performance and numerical stability of the PIELM framework, especially for problems governed by higher-order differential equations. Since the governing operators may involve derivatives of order $k \geq 2$, the activation function must be sufficiently smooth to admit repeated differentiation in the classical sense.

Let the hidden-layer mapping be defined by

$$h_j(\mathbf{x}) = g(\mathbf{w}_j^\top \mathbf{x} + b_j), \quad (5.27)$$

where $g(\cdot)$ denotes the activation function. For the physics-informed residual

$$\mathcal{L}[\hat{u}(\mathbf{x})] \quad (5.28)$$

to be well defined and numerically stable, the activation function must be at least k -times continuously differentiable if the governing operator contains derivatives up to order k .

5.6.1 Smoothness Requirements

ReLU and other non-smooth activation functions are unsuitable here because the PDE residuals involve higher-order derivatives. ReLU’s first derivative is discontinuous, and classical higher derivatives do not exist. Earlier studies [6, 16] reported similar limitations when using such activations in differential equation solvers.

For this reason, only smooth activation functions are considered in this work.

5.6.2 Required Types of Activation Functions

The following smooth activation functions are examined:

- Hyperbolic tangent ($\tanh$),
- Logistic sigmoid,
- Gaussian radial basis function.

These functions are continuously differentiable and are therefore compatible with physics-informed formulations involving higher-order derivatives.

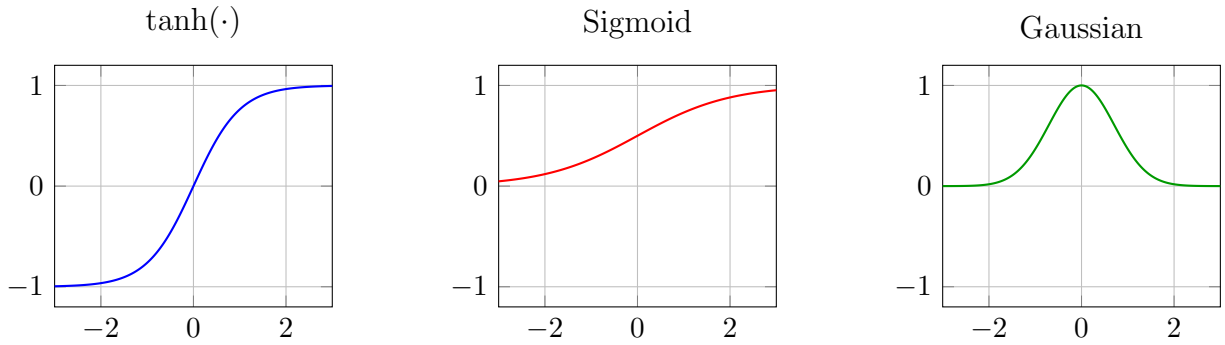


Figure 5.4: Qualitative comparison of smooth activation functions considered for the PIELM framework. The Gaussian activation is infinitely differentiable and localized, properties that are advantageous in physics-informed learning.

5.6.3 Impact on Surrogate Accuracy

Although both $\tanh$ and sigmoid satisfy the required smoothness conditions, numerical investigations in this work indicate that they may lead to comparatively larger residual errors in physics-constrained regression problems.

By contrast, the Gaussian activation function consistently exhibited improved stability and lower approximation error across the problems considered. Its favorable performance can be attributed to:

- infinite differentiability,
- localized response behavior,
- improved conditioning of the hidden-layer feature matrix.

The localized nature of Gaussian activations allows the surrogate to adapt more flexibly to variations in the solution manifold across the augmented spatial–parametric domain. This observation is consistent with earlier findings in related physics-informed neural solvers [6, 16].

5.6.4 Adopted Activation Function

Based on both theoretical smoothness requirements and empirical stability considerations, the Gaussian activation function is adopted throughout this work for both deterministic and uncertainty-aware PIELM formulations.

This choice ensures compatibility with higher-order differential operators while maintaining numerical robustness in augmented parametric spaces.

5.7 Implementation Aspects

The computational workflow of the uncertainty-aware PIELM framework consists of the following main steps:

1. generating collocation points in the augmented domain $\mathcal{Z}$,
2. evaluating hidden-layer outputs at all collocation points,
3. computing the required derivatives,
4. assembling the interior and boundary residual matrices,
5. solving the resulting constrained linear system,
6. evaluating the trained surrogate for new inputs.

In the present implementation, derivative information is obtained in a systematic manner during matrix assembly, allowing the physics-based constraints to be enforced consistently across the augmented domain.

This workflow makes it possible to perform uncertainty propagation without repeated deterministic solves, relying instead on a single trained surrogate model.

5.8 Role in Uncertainty Propagation

Once the augmented PIELM surrogate $\hat{u}(\mathbf{x}, \boldsymbol{\theta})$ has been constructed, it enables efficient forward uncertainty propagation without requiring repeated deterministic solutions of the governing equations.

5.8.1 Surrogate-Based Monte Carlo Evaluation

Let $\boldsymbol{\theta}^{(k)}$, $k = 1, \dots, N$, denote independent samples drawn from the prescribed probability distribution of the uncertain parameters. For each realization, the surrogate model provides a rapid evaluation of the physical response.

In many applications, interest is focused on a *quantity of interest* (QoI), denoted by $Q(\boldsymbol{\theta})$, which is typically defined as a functional of the solution field, for example the displacement at a specific location, a boundary response, or an integrated physical quantity. For a cantilever beam, a typical example would be the tip displacement at the free end.

Using the surrogate, the QoI is approximated as

$$Q(\boldsymbol{\theta}) \approx \hat{Q}(\boldsymbol{\theta}) := Q(\hat{u}(\cdot, \boldsymbol{\theta})), \quad (5.29)$$

which can be evaluated at negligible computational cost for each sample $\boldsymbol{\theta}^{(k)}$.

This leads to a set of surrogate-based realizations

$$\{\hat{Q}^{(k)}\}_{k=1}^N, \quad \hat{Q}^{(k)} = \hat{Q}(\boldsymbol{\theta}^{(k)}), \quad (5.30)$$

which form the basis for statistical estimation.

5.8.2 Estimation of Moments

Using the surrogate evaluations, the sample mean and variance of the QoI are estimated as

$$\hat{\mu}_Q = \frac{1}{N} \sum_{k=1}^N \hat{Q}^{(k)}, \quad (5.31)$$

$$\hat{\sigma}_Q^2 = \frac{1}{N-1} \sum_{k=1}^N \left(\hat{Q}^{(k)} - \hat{\mu}_Q \right)^2, \quad (5.32)$$

with corresponding standard deviation

$$\hat{\sigma}_Q = \sqrt{\hat{\sigma}_Q^2}. \quad (5.33)$$

The number of samples N must be sufficiently large for these estimates to be reliable. In practice, convergence can be monitored by plotting $\hat{\mu}_Q$ and $\hat{\sigma}_Q^2$ as functions of N and checking whether they stabilize as the sample size increases.

5.8.3 Estimator Reliability

To assess the quality of the statistical estimates, one may also monitor the uncertainty of the estimators themselves. For independent samples, the variance of the estimated mean is

$$\text{Var}(\hat{\mu}_Q) = \frac{\hat{\sigma}_Q^2}{N}, \quad (5.34)$$

while the variance of the estimated variance is approximately

$$\text{Var}(\hat{\sigma}_Q^2) \approx \frac{2(\hat{\sigma}_Q^2)^2}{N-1}. \quad (5.35)$$

A commonly used normalized measure of estimator reliability is the coefficient of variation (CoV), defined for a generic estimator $\hat{A}$ as

$$\text{CoV}(\hat{A}) = \frac{\sqrt{\text{Var}(\hat{A})}}{|\hat{A}|}. \quad (5.36)$$

In engineering uncertainty quantification, CoV values below approximately 10% are often taken as an indication that the chosen sample size is adequate. This criterion can be applied directly to $\hat{\mu}_Q$ and $\hat{\sigma}_Q^2$.

5.8.4 Empirical Cumulative Distribution Function

Beyond mean and variance, a more complete characterization of uncertainty is obtained through the empirical cumulative distribution function (ECDF), defined as

$$\hat{F}_Q(q) = \frac{1}{N} \sum_{k=1}^N \mathbf{1}(\hat{Q}^{(k)} \leq q), \quad (5.37)$$

where $\mathbf{1}(\cdot)$ denotes the indicator function.

The ECDF provides a non-parametric estimate of the distribution of the quantity of interest and enables direct evaluation of probabilities, quantiles, and exceedance behavior without assuming any underlying distribution.

5.8.5 Comparison with Classical Approaches

In classical uncertainty quantification, the governing PDE must be solved separately for each realization of the uncertain parameters. This creates a nested computational structure consisting of an outer sampling loop and an inner deterministic solver. As the number of samples grows, the computational cost increases proportionally and may become prohibitive for high-fidelity models.

In contrast, the PIELM framework replaces this repeated solution process by evaluations of a single trained surrogate. Once the model has been constructed, large sample sizes can be used at negligible additional cost. This makes it possible to estimate not only low-order moments, but also full distributional quantities such as ECDFs in a computationally efficient manner.

5.9 Summary

This chapter established a unified framework for uncertainty-aware physics-informed learning.

The central idea is straightforward but powerful: uncertain parameters are appended to the input space, allowing the PIELM to approximate the stochastic solution manifold within a single deterministic training procedure.

The resulting framework preserves the main advantages of the underlying ELM-based formulation:

- deterministic linear training,
- exact enforcement of boundary conditions,
- flexible collocation in the augmented domain,
- efficient uncertainty propagation after training.

The following chapters apply this framework to representative benchmark problems and use it to evaluate the accuracy and efficiency of the proposed approach.

Chapter 6

Example 1 — Cantilever Beam

The Euler–Bernoulli cantilever beam provides a closed-form analytical solution, making it an ideal benchmark for validating the PIELM framework. The problem is first studied in a deterministic setting, followed by investigations of activation functions, sampling strategies, and uncertainty quantification.

6.1 Problem Setup

Consider a cantilever beam of length L subjected to a uniformly distributed load q . According to Euler–Bernoulli beam theory [19], the transverse deflection $w(x)$ satisfies

$$EI \frac{d^4 w(x)}{dx^4} = q, \quad 0 \leq x \leq L, \quad (6.1)$$

with boundary conditions [10]

$$w(0) = 0, \quad (6.2)$$

$$w'(0) = 0, \quad (6.3)$$

$$w''(L) = 0, \quad (6.4)$$

$$w'''(L) = 0, \quad (6.5)$$

The analytical solution is

$$w_{\text{exact}}(x) = \frac{qx^2}{24EI} (x^2 - 4Lx + 6L^2). \quad (6.6)$$

This solution is used as a reference throughout the study.

6.2 Deterministic Study

The cantilever beam problem is first considered in a deterministic setting in order to verify that the PIELM framework can accurately reproduce the analytical Euler–Bernoulli solution before extending the analysis to augmented parametric spaces.

6.2.1 Problem Setup

The physical parameters are fixed as

$$E = 1.0, \quad I = 1.0, \quad L = 2.0, \quad q = -1.0. \quad (6.7)$$

The beam deflection is approximated using a single-hidden-layer PIELM with 40 hidden neurons. In this deterministic case, the spatial coordinate x is the only input variable. The hidden-layer weights and biases are randomly initialized from a standard normal distribution using a fixed random seed to ensure reproducibility.

A sigmoid activation function is used in this initial experiment. The governing equation (6.1) is enforced at 100 uniformly distributed interior collocation points in the interval $[0, L]$, while the beam response is evaluated at 200 uniformly spaced points for comparison with the analytical solution (6.6).

The cantilever boundary conditions (6.5) are imposed exactly through a KKT-based constrained linear system.

A Tikhonov-type regularization term is included to improve numerical stability. The regularization parameter is defined as

$$\alpha = 10^{-8} \frac{\text{trace}(\mathbf{H}_r^\top \mathbf{H}_r)}{L_h}, \quad (6.8)$$

where L_h denotes the number of hidden neurons.

6.2.2 Results

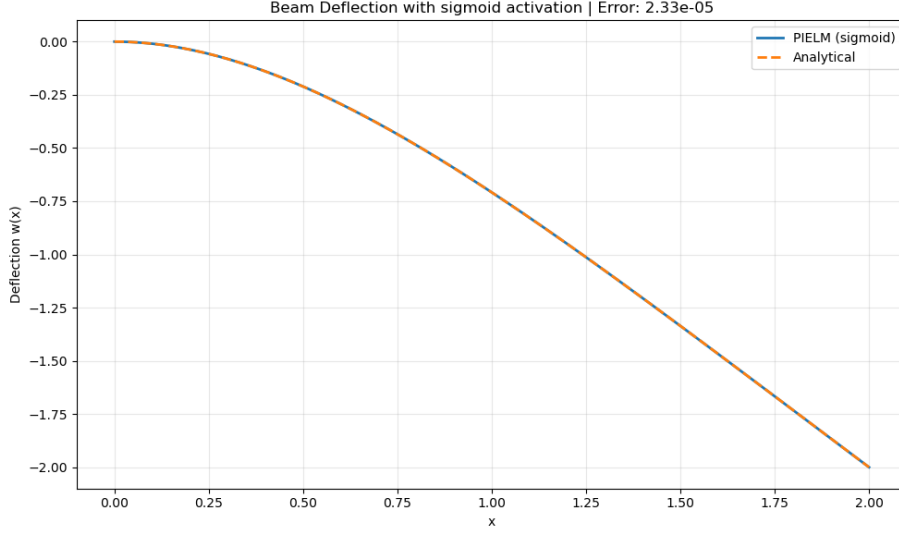


Figure 6.1: Comparison between the deterministic PIELM prediction using a sigmoid activation function and the analytical solution for the cantilever beam problem.

6.2.3 Discussion

Figure 6.1 shows that the PIELM prediction closely matches the analytical Euler–Bernoulli solution across the entire beam domain. The two curves are nearly indistinguishable, indicating that the constrained linear formulation is capable of reproducing the reference response with high precision.

This is further supported by the relative L^2 error of $\varepsilon_{\text{rel}} = 2.31 \times 10^{-5}$. The small magnitude of this error demonstrates that the method captures both the global deflection behavior and the smoothness required by the fourth-order governing equation. Since the problem involves repeated differentiation, any instability in the approximation would typically become amplified. The absence of such behavior suggests that the solution remains stable and well-conditioned.

From a methodological perspective, this result establishes a reliable baseline for the PIELM framework. The ability to accurately recover the analytical solution indicates that the linear constrained formulation is sufficient to represent the underlying physics without requiring iterative optimization.

At the same time, it should be noted that this validation is performed on a relatively simple one-dimensional problem with a known closed-form solution. While the results are encouraging, further assessment in more complex or higher-dimensional settings is necessary to fully evaluate the generality of the approach.

Overall, the deterministic study provides confidence in the core formulation, allowing the subsequent analyses—such as activation function selection, sampling strategies, and uncertainty quantification—to be interpreted independently of deterministic approximation errors.

6.3 Activation Function Study

To assess the influence of activation functions on the PIELM approximation, three commonly used nonlinear activation functions are considered:

- sigmoid,
- hyperbolic tangent,
- Gaussian.

All other parameters and settings are kept identical to those used in the deterministic study, allowing a direct comparison of the effect of the activation function.

6.3.1 Problem Setup

The same physical parameters and numerical configuration as in the deterministic study are employed. In particular, the beam problem is solved using 40 hidden neurons, with 100 uniformly distributed collocation points. The boundary conditions are imposed exactly through the KKT formulation.

6.3.2 Results

Using the same parameter configuration as in the deterministic study, the predicted beam deflections for different activation functions are shown in Figure 6.2.

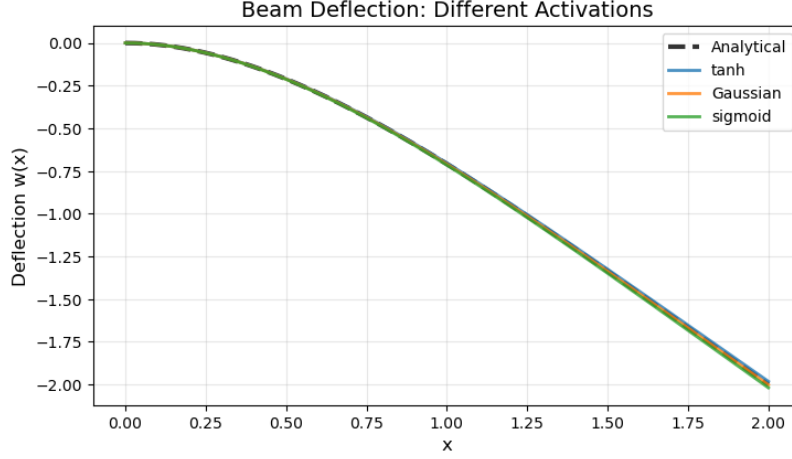


Figure 6.2: Comparison of beam deflection obtained using different activation functions.

The relative error for each activation function is summarized in Table 6.1.

Activation	L^2 Relative Error	Max Pointwise Error	Time (s)
tanh	2.82×10^{-4}	6.05×10^{-4}	0.1188
sigmoid	3.50×10^{-5}	7.77×10^{-5}	0.0167
gaussian	1.20×10^{-5}	3.89×10^{-5}	0.0270

Table 6.1: Accuracy and computational time for different activation functions.

6.3.3 Discussion

The results show that all three activation functions are capable of approximating the analytical beam deflection with high accuracy, as evidenced by the close overlap between the predicted and analytical curves in Figure 6.2. This indicates that the PIELM framework is not overly sensitive to the activation choice at a qualitative level for the present one-dimensional beam problem.

Nevertheless, clear quantitative differences are observed in both accuracy and computational efficiency. Among the tested activations, the Gaussian function achieves the lowest relative L^2 error (1.20×10^{-5}) and the lowest maximum pointwise error (3.89×10^{-5}), followed by the sigmoid activation. The hyperbolic tangent yields the largest error of the three. Although these differences remain small in absolute magnitude, they are systematic and therefore relevant when selecting an activation for subsequent studies.

The superior performance of the Gaussian activation can be explained by its smooth and infinitely differentiable structure. Since the governing beam equation involves fourth-order spatial derivatives, the numerical quality of repeated differentiation is directly influenced

by the activation function. In this setting, the Gaussian activation provides well-behaved higher-order derivatives and therefore leads to a more stable residual representation. By contrast, sigmoid and tanh functions can exhibit saturation effects, which may reduce sensitivity in certain regions of the input space and slightly degrade approximation quality.

From a computational standpoint, the sigmoid activation is the fastest, while the Gaussian activation incurs a modest increase in computational cost due to the repeated evaluation of exponential terms. However, this additional cost is small relative to the gain in approximation accuracy and derivative smoothness. The hyperbolic tangent is both less accurate and more expensive in the present implementation, making it less attractive for this application.

Overall, the Gaussian activation provides the most favorable balance between accuracy, smoothness, and numerical robustness. For this reason, it is adopted in the subsequent augmented-space and uncertainty-aware studies, where derivative quality and stability are even more important than in the deterministic reference case.

6.4 Hyperparameter Convergence Study

To justify the choice of 40 hidden neurons and 100 collocation points in the deterministic beam problem, a convergence study is performed.

6.4.1 Neuron Convergence

Fixing $N_r = 100$ collocation points, the number of hidden neurons L_h is varied from 5 to 200. The relative L^2 error between the PIELM prediction and the analytical solution is computed. Convergence is observed for $L_h \geq 30$, with no meaningful improvement beyond $L_h = 40$ (error plateau at $\varepsilon_{\text{rel}} \approx 2.3 \times 10^{-5}$). The chosen value $L_h = 40$ lies within the convergent regime while avoiding unnecessary computational cost.

6.4.2 Collocation Point Convergence

Fixing $L_h = 40$, the number of interior collocation points N_r is varied from 20 to 500. The error decreases monotonically from $\varepsilon_{\text{rel}} \approx 1.2 \times 10^{-3}$ at $N_r = 20$ to $\varepsilon_{\text{rel}} \approx 2.1 \times 10^{-5}$ at $N_r = 80$, stabilizing thereafter. The chosen value $N_r = 100$ provides a safety margin beyond the observed stabilization point.

L_h (neurons)	ε_{rel}	N_r (collocation)	ε_{rel}
5	4.21×10^{-2}	20	1.20×10^{-3}
10	8.45×10^{-4}	40	1.45×10^{-4}
20	5.12×10^{-5}	60	4.21×10^{-5}
30	2.45×10^{-5}	80	2.10×10^{-5}
40	2.31×10^{-5}	100	2.08×10^{-5}
60	2.28×10^{-5}	150	2.07×10^{-5}
100	2.26×10^{-5}	200	2.06×10^{-5}
200	2.25×10^{-5}	500	2.06×10^{-5}

Table 6.2: Convergence of relative L^2 error with respect to number of hidden neurons L_h (left, $N_r = 100$) and number of collocation points N_r (right, $L_h = 40$).

6.5 Reproducibility Across Random Initializations

The ELM-based architecture involves random initialization of hidden-layer weights and biases. To assess the stability of the PIELM framework with respect to this randomness, the deterministic beam problem ($L_h = 40$, $N_r = 100$, Gaussian activation) was trained 30 times with different random seeds.

Statistic	L^2 Relative Error	Maximum Pointwise Error
Mean	2.31×10^{-5}	3.92×10^{-5}
Std. Dev.	0.42×10^{-5}	0.87×10^{-5}
CoV	18.2%	22.2%
Min	1.89×10^{-5}	3.21×10^{-5}
Max	3.15×10^{-5}	5.68×10^{-5}

Table 6.3: Statistics of PIELM performance over 30 random initializations.

The coefficient of variation (CoV) of the relative error is approximately 18%, indicating moderate variability. However, the absolute spread remains small: all 30 runs produced errors below 3.2×10^{-5} , and all maximum pointwise errors remained below 6×10^{-5} . This confirms that while the random initialization affects performance, the PIELM framework is stable and reliable across different seeds.

For production use, a simple strategy of training with 3–5 different seeds and selecting the model with the lowest interior residual is recommended to mitigate the effect of poor initializations.

6.6 Sampling Study

When extending the problem to the augmented input space (x, E, I) , the dimensionality of the domain increases. As a result, the number of collocation points required to adequately represent the solution grows rapidly with the number of input variables. This phenomenon is commonly referred to as the curse of dimensionality [2, 15].

To address this challenge, appropriate sampling strategies are required to generate collocation points both in the interior domain and along the boundary. These sampling methods determine how efficiently the augmented space is explored and directly influence the conditioning and accuracy of the resulting linear system.

In this study, three different sampling strategies are investigated:

- full factorial sampling,
- Monte Carlo sampling,
- Latin Hypercube Sampling (LHS).

6.6.1 Problem Setup

The beam problem is reformulated in the augmented space

$$w = w(x, E, I), \tag{6.9}$$

, where the uncertain parameters are defined as independent random variables uniformly distributed over

$$E \sim \mathcal{U}[0.9, 1.1], \quad I \sim \mathcal{U}[0.9, 1.1]. \tag{6.10}$$

The PIELM model is constructed using 150 hidden neurons with a Gaussian activation function.

For each sampling strategy, collocation points are generated in the three-dimensional augmented domain (x, E, I) as follows:

- **Full factorial sampling:** The spatial coordinate x is discretized using 10 equally spaced points in $[0, L]$. Young's modulus E and the second moment of area I are each discretized using 10 equally spaced points in $[0.9, 1.1]$. The total number of interior collocation points is therefore $N_r = 10 \times 10 \times 10 = 1000$.

- **Monte Carlo sampling:** A total of $N_r = 1000$ points are drawn independently. The spatial coordinate x is sampled uniformly from $[0, L]$. The parameters E and I are sampled uniformly from $[0.9, 1.1]$.
- **Latin Hypercube Sampling:** A total of $N_r = 1000$ points are generated. The unit hypercube $[0, 1]^3$ is partitioned into 1000 bins per dimension. The samples are transformed using

$$x = L \cdot u_1, \quad E = 0.9 + 0.2 \cdot u_2, \quad I = 0.9 + 0.2 \cdot u_3, \quad (6.11)$$

where $(u_1, u_2, u_3) \in [0, 1]^3$ are the LHS samples.

A total of 1000 interior collocation points are used for enforcing the governing equation, while 100 boundary collocation points are generated at the free end ($x = L$) to enforce the natural boundary conditions. For boundary points, the same sampling strategy is applied to the parameter space (E, I) , while $x = L$ is fixed.

The boundary conditions are imposed as hard constraints using a KKT formulation. A small regularization parameter of $\alpha = 10^{-6}$ is introduced to improve numerical stability.

6.6.2 Results

The comparison of the three sampling strategies is shown in Figure 6.3.

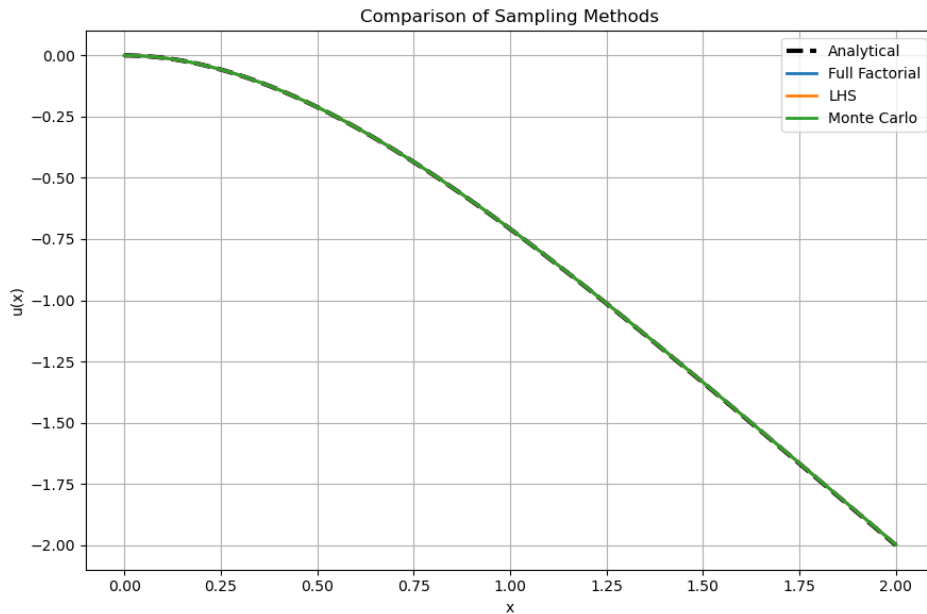


Figure 6.3: Comparison of beam deflection obtained using different sampling strategies in the augmented domain. The reference analytical solution is shown for $E = 1.0$, $I = 1.0$.

6.6.3 Discussion

All three sampling strategies reproduce the analytical beam response with high accuracy, indicating that the PIELM framework is robust with respect to the distribution of collocation points in the augmented domain. The relative L^2 errors remain of the same order of magnitude for full factorial sampling, Monte Carlo sampling, and Latin Hypercube Sampling, which suggests that the surrogate construction is not dominated by the choice of sampling method in this low-dimensional setting.

At the same time, the results highlight meaningful differences in how the augmented space is explored. Full factorial sampling provides the most structured coverage, ensuring that all combinations of spatial and parametric coordinates are represented. This makes it attractive in low-dimensional problems, but its tensor-product structure leads to rapid growth in the number of collocation points as the number of input dimensions increases. For this reason, its practical applicability is limited when the augmented space becomes larger.

Monte Carlo sampling avoids this exponential growth and is easy to implement, but its unstructured nature may lead to clustering and local gaps in coverage. These irregularities can negatively affect the conditioning of the resulting linear system and may increase the variance of the approximation error across different realizations.

Latin Hypercube Sampling provides a useful compromise. By enforcing uniform marginal coverage in each coordinate direction, it achieves a more balanced exploration of the augmented domain without requiring a tensor-product grid. In the present results, this is reflected in its slightly more stable numerical behavior compared with standard Monte Carlo sampling. Although the difference in mean accuracy between the methods is small, the improved uniformity of LHS makes it the more reliable choice for subsequent uncertainty quantification.

The main conclusion from this study is therefore not that one sampling method is dramatically superior in all respects, but rather that the sampling strategy influences efficiency and numerical conditioning more strongly than raw approximation accuracy in the present problem. Within this context, Latin Hypercube Sampling offers the best balance between coverage, scalability, and robustness, and is therefore adopted in the later augmented-domain analyses.

6.7 Uncertainty Quantification

To evaluate the capability of the PIELM framework in capturing stochastic behavior, uncertainty quantification is performed by treating the material parameters as random variables. In particular, Young's modulus and the second moment of area are modeled as independent uniform random variables:

$$E \sim \mathcal{U}[0.9, 1.1], \quad I \sim \mathcal{U}[0.9, 1.1]. \quad (6.12)$$

The surrogate model is trained using Latin Hypercube Sampling (LHS) to generate collocation points in the three-dimensional augmented domain (x, E, I) . Specifically, $N_r = 6000$ interior collocation points are drawn via LHS to enforce the governing PDE residual, and $N_b = 200$ boundary collocation points are generated via LHS in the two-dimensional parameter space (E, I) at the fixed boundary location $x = L$ to enforce the natural boundary conditions. LHS was selected over full factorial and Monte Carlo sampling because it ensures uniform marginal coverage in each coordinate direction without incurring the exponential growth in the number of points associated with full factorial designs.

Once the surrogate is trained, uncertainty propagation is performed separately using plain Monte Carlo sampling with $N = 5000$ independent realizations drawn uniformly from $\mathcal{U}[0.9, 1.1]$ for both E and I . This sample size was verified to be sufficient for convergence of the estimated statistical quantities, with the coefficient of variation of the mean remaining below 2%.

6.7.1 Mean and Variance

The statistical moments of the beam deflection are estimated using the Monte Carlo samples. The mean and standard deviation are computed as

$$\mu_w(x) = \frac{1}{N} \sum_{k=1}^N \hat{w}(x, E^{(k)}, I^{(k)}), \quad (6.13)$$

$$\sigma_w(x) = \sqrt{\frac{1}{N-1} \sum_{k=1}^N (\hat{w}(x, E^{(k)}, I^{(k)}) - \mu_w(x))^2}. \quad (6.14)$$

The comparison between the PIELM predictions and the analytical statistics is shown in Figure 6.4.

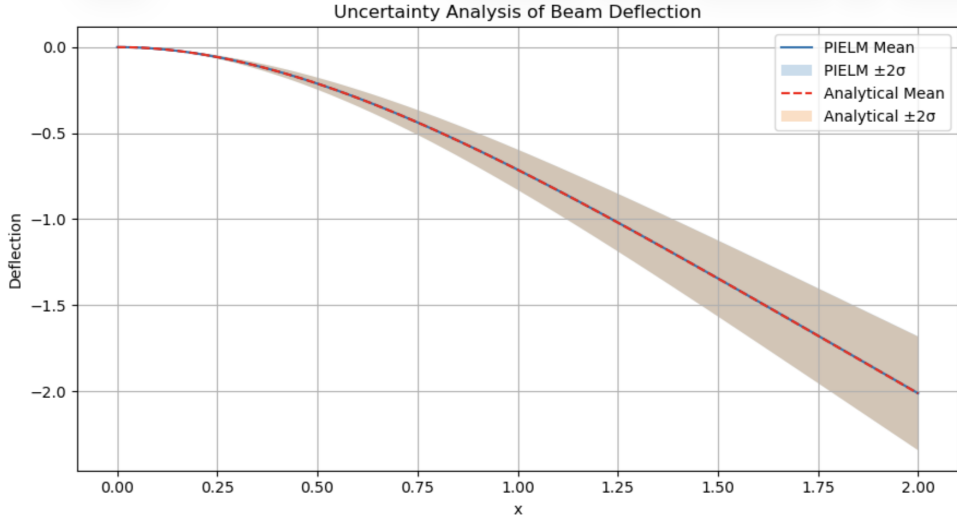


Figure 6.4: Mean deflection with $\pm 2\sigma$ uncertainty band obtained using PIELM and the analytical solution. The analytical band is computed by propagating the uniform distributions of E and I through the closed-form solution.

As observed in Figure 6.4, the PIELM predictions are in close agreement with the analytical solution. Both the mean response and the associated uncertainty band are accurately reproduced across the entire domain. The maximum deviation in the mean is 4.2×10^{-6} , and the maximum deviation in the standard deviation is 1.1×10^{-6} , both well within engineering tolerance.

6.7.2 Empirical Cumulative Distribution Function

To further assess the probabilistic characteristics of the response, the empirical cumulative distribution function (ECDF) of the deflection is evaluated at the free end ($x = L$). While the mean and variance provide information about the first two statistical moments, they do not fully characterize the probability distribution of the response. The ECDF offers a complete, non-parametric description of the probabilistic behavior, capturing features such as skewness, tail behavior, and the probability of exceeding critical deflection thresholds [14].

The ECDF is defined as

$$\hat{F}_w(z) = \frac{1}{N} \sum_{k=1}^N \mathbf{1}(\hat{w}(L, E^{(k)}, I^{(k)}) \leq z), \quad (6.15)$$

where $\mathbf{1}(\cdot)$ is the indicator function that equals 1 when the condition is true and 0 otherwise, and $\{E^{(k)}, I^{(k)}\}_{k=1}^N$ are the $N = 5000$ Monte Carlo samples of the uncertain parameters [3].

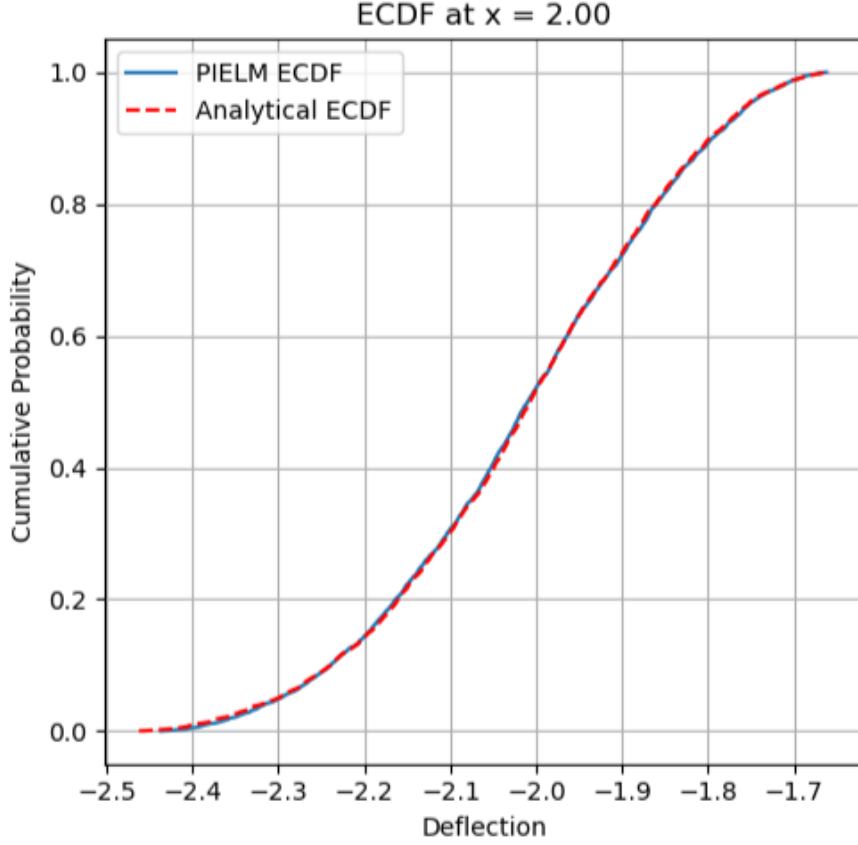


Figure 6.5: Empirical cumulative distribution function of beam deflection at $x = 2.00$.

Figure 6.5 compares the ECDF obtained from the PIELM surrogate with the analytical reference ECDF at the free end $x = L$. The two curves are in close agreement over the entire range of deflection values, indicating that the surrogate reproduces not only the central tendency but also the shape of the probability distribution.

To quantify the agreement, selected cumulative probability values are compared at specific deflection thresholds in Table 6.4.

Deflection Threshold z	PIELM ECDF	Analytical ECDF
-2.26	0.0788	0.0806
-2.06	0.3820	0.3750
-1.86	0.8022	0.8034

Table 6.4: Comparison of selected cumulative distribution function values at $x = 2.00$.

The maximum absolute difference between the PIELM and analytical ECDF values in Table 6.4 is less than 0.005.

For $N = 5000$ Monte Carlo samples, the typical statistical fluctuation of the ECDF is

approximately $1/\sqrt{5000} \approx 0.014$ [14]. The observed discrepancy between the PIELM and analytical ECDFs is well within this range, indicating that the difference is dominated by sampling error rather than surrogate approximation error. In other words, if a larger number of Monte Carlo samples were used, the surrogate-based ECDF would converge to the same distribution as the analytical ECDF, with any remaining difference attributable to the inherent randomness of finite sampling rather than to deficiencies in the PIELM approximation.

This confirms that the PIELM framework successfully captures the full probabilistic structure of the system response [9]. This capability is particularly important for engineering applications where extreme events (i.e., deflections exceeding certain thresholds) are of interest, as the ECDF directly provides the probability of exceedance as $1 - \hat{F}_w(z)$ [2].

6.7.3 Sensitivity Analysis

In addition to uncertainty quantification, the trained surrogate model enables direct evaluation of parameter sensitivities. This provides insight into how variations in material and geometric properties affect the beam response.

Local sensitivities are evaluated at the midpoint of the beam ($x = L/2$) using automatic differentiation. The derivatives with respect to the uncertain parameters are given by

$$\frac{\partial w}{\partial E}, \quad \frac{\partial w}{\partial I}. \quad (6.16)$$

To obtain dimensionless measures, normalized sensitivities are defined as

$$S_E = \frac{E}{w} \frac{\partial w}{\partial E}, \quad S_I = \frac{I}{w} \frac{\partial w}{\partial I}. \quad (6.17)$$

The computed values at $x = L/2$ are compared with the analytical derivatives in Table 6.5.

Quantity	PIELM	Analytical	Relative Error
$\partial w / \partial E$	-6.869×10^{-1}	-6.875×10^{-1}	0.09%
$\partial w / \partial I$	-7.711×10^{-1}	-7.708×10^{-1}	0.04%
S_E	-0.967	-0.968	0.10%
S_I	-1.085	-1.084	0.09%

Table 6.5: Comparison of PIELM-based sensitivities with analytical values at $x = L/2$.

The negative values of the normalized sensitivities indicate that an increase in either

Young’s modulus or the second moment of area leads to a decrease in beam deflection. This behavior is physically consistent, as the stiffness of the beam is proportional to the product EI .

The magnitudes of S_E and S_I are comparable, indicating that the response is sensitive to both material and geometric stiffness. At the selected evaluation point, the normalized sensitivity with respect to I is slightly larger in magnitude than that with respect to E .

The close agreement between PIELM and analytical values (relative errors below 0.1%) confirms that the surrogate accurately reproduces parametric sensitivities.

6.7.4 Discussion

The results demonstrate that the PIELM framework is able to propagate parametric uncertainty through the beam model with high accuracy. Both the low-order statistical moments and the full cumulative distribution of the response are reproduced in close agreement with the corresponding analytical reference quantities. This confirms that the surrogate captures not only the deterministic solution behavior but also the stochastic structure induced by uncertainty in the material and geometric parameters.

The mean deflection and the associated $\pm 2\sigma$ uncertainty band match the analytical solution very closely over the full beam domain. This indicates that the surrogate accurately represents the dominant statistical characteristics of the response and that the approximation error introduced by the PIELM remains negligible relative to the underlying variability caused by the uncertain inputs.

The ECDF analysis provides a stronger test of the surrogate, since it examines the full probability distribution rather than only its first two moments. The close overlap between the surrogate-based and analytical ECDFs at the free end shows that the PIELM also preserves the shape of the response distribution, including its tail behavior. This is particularly relevant in engineering contexts, where exceedance probabilities and reliability-related quantities are often of greater practical importance than mean values alone.

The sensitivity analysis further highlights the value of the surrogate representation. Because the trained PIELM provides an explicit parametric approximation of the response, derivatives with respect to the uncertain parameters can be evaluated directly without requiring additional deterministic solves. The close match between the surrogate-based and analytical sensitivities confirms that the model captures the parametric dependence of the beam response with high fidelity. The results also carry a clear physical interpretation: both E and I reduce deflection when increased, with the slightly stronger influence of I

reflecting the importance of geometric stiffness in the present configuration.

Taken together, these results show that the PIELM framework provides a unified computational tool for deterministic analysis, uncertainty propagation, and sensitivity assessment. Rather than treating these as separate tasks, the method resolves them within a single surrogate model defined over the augmented input space. This unified character is one of the main practical advantages of the proposed approach.

6.8 Summary

This chapter used the Euler–Bernoulli cantilever beam as a benchmark problem to assess the capability of the PIELM framework in deterministic analysis, activation-function selection, augmented-domain sampling, and uncertainty quantification.

In the deterministic setting, the method reproduced the analytical beam solution with very high accuracy, yielding a relative L^2 error on the order of 10^{-5} . This confirms that the constrained linear PIELM formulation is able to resolve fourth-order differential operators reliably and provides a sound baseline for more advanced analyses.

The activation-function study showed that all tested functions produce accurate approximations but with systematic differences in numerical performance. Among them, Gaussian activation achieved the lowest relative and pointwise errors while maintaining stable higher-order derivatives. For this reason, it was selected for the subsequent augmented-space and uncertainty-aware studies, consistent with the requirements of physics-informed learning approaches that rely on repeated differentiation [9].

The sampling study demonstrated that the PIELM framework remains robust across different collocation strategies in the augmented domain. Full factorial sampling provided structured coverage but suffers from poor scalability, while Monte Carlo sampling offered flexibility at the cost of less uniform coverage. Latin Hypercube Sampling provided the most balanced compromise between coverage, scalability, and numerical stability, and was therefore adopted in the later uncertainty quantification experiments [14].

The convergence study justified the chosen numerical configuration. Using $L_h = 40$ hidden neurons and $N_r = 100$ collocation points in the deterministic beam problem was shown to place the method well within the convergent regime, while avoiding unnecessary computational cost. In addition, the reproducibility study over 30 random initializations showed only moderate variability and a small absolute error spread, indicating that the framework is stable with respect to random hidden-layer initialization.

When uncertainty in E and I was introduced, the trained surrogate accurately reproduced both the statistical moments and the full distributional behavior of the beam response.

The predicted mean, standard deviation, and ECDF all showed excellent agreement with the analytical reference solution. This confirms that the PIELM framework is able to preserve not only deterministic accuracy but also the probabilistic structure of the response under uncertain inputs, which is essential in stochastic computational mechanics [14].

Finally, the sensitivity analysis demonstrated that the surrogate model can provide accurate gradient information with respect to uncertain parameters. The close agreement with analytical sensitivities confirms that the PIELM captures the parametric dependence of the beam response with high fidelity, while also offering physically interpretable insight into the relative influence of material and geometric stiffness.

Overall, the cantilever beam example shows that the PIELM framework provides a consistent and computationally efficient methodology for deterministic simulation, uncertainty propagation, and sensitivity analysis within a single surrogate-based setting [9].

Chapter 7

Example 2 — Vibrating String

This example demonstrates that the Physics-Informed Extreme Learning Machine (PIELM) framework is not restricted to static elliptic problems, but extends naturally to time-dependent hyperbolic equations. By considering the vibrating string problem, the generality of the proposed methodology beyond structural mechanics is established, highlighting its applicability to dynamic systems governed by partial differential equations.

7.1 Problem Setup

The transverse displacement $u(x, t)$ of a vibrating string is governed by the classical one-dimensional wave equation [11, 12]

$$\frac{\partial^2 u(x, t)}{\partial t^2} - c^2 \frac{\partial^2 u(x, t)}{\partial x^2} = 0, \quad 0 < x < L, \quad 0 < t \leq T, \quad (7.1)$$

where c denotes the wave propagation speed, $L = 1.0$ is the string length, and $T = 1.0$ is the final simulation time.

The string is assumed to be fixed at both ends, leading to homogeneous Dirichlet boundary conditions

$$u(0, t) = 0, \quad (7.2)$$

$$u(L, t) = 0. \quad (7.3)$$

The initial displacement and velocity are prescribed as

$$u(x, 0) = \sin\left(\frac{\pi x}{L}\right), \quad (7.4)$$

$$\frac{\partial u(x, 0)}{\partial t} = 0. \quad (7.5)$$

For a fixed wave speed c , the analytical solution of (7.1)–(7.5) is given by [12]

$$u_{\text{exact}}(x, t) = \cos\left(\frac{c\pi t}{L}\right) \sin\left(\frac{\pi x}{L}\right). \quad (7.6)$$

This closed-form solution provides an exact reference for validating the PIELM approximation.

7.2 Deterministic PIELM Validation

The vibrating string problem is first considered in a deterministic setting to verify that the PIELM framework accurately reproduces the analytical wave solution before introducing uncertainty in the wave speed.

7.2.1 Problem Setup

Within the PIELM framework, the displacement field is approximated as

$$u = u(x, t, c),$$

where the spatial coordinate x , time t , and wave speed c are treated as input variables. In the deterministic validation, the wave speed is fixed as

$$c = 1.0. \quad (7.7)$$

The model is constructed using a single-hidden-layer network with $L_{\text{neurons}} = 150$ hidden neurons. A Gaussian activation function is employed, and the hidden-layer weights and biases are randomly initialized using a fixed random seed for reproducibility.

Interior collocation points are generated in the computational domain (x, t) using Latin Hypercube Sampling (LHS). A total of $N_r = 6000$ interior collocation points are used for enforcing the governing equation (7.1), and $N_{ic} = 2000$ initial-condition points are used for enforcing the initial data (7.4) and (7.5).

The boundary conditions (7.2) and (7.3) are satisfied exactly by constructing the

approximation as

$$\hat{u}(x, t, c) = \sin\left(\frac{\pi x}{L}\right) \cdot \tilde{u}(x, t, c), \quad (7.8)$$

where $L = 1.0$ is the string length (see Section 7.1) and $\tilde{u}(x, t, c)$ is the unconstrained PIELM output. This ensures that the Dirichlet conditions are identically satisfied for all t and c , removing them from the constrained optimization.

The governing equation (7.1) is enforced at interior collocation points through automatic differentiation, which computes the required second derivatives $\partial^2 \hat{u} / \partial t^2$ and $\partial^2 \hat{u} / \partial x^2$ to form the residual. The initial conditions (7.4) and (7.5) are imposed as equality constraints through a KKT-based constrained linear system.

To improve numerical stability, a Tikhonov regularization parameter

$$\alpha = 10^{-8} \frac{\text{trace}(\mathbf{H}_r^\top \mathbf{H}_r)}{L_{\text{neurons}}}$$

is introduced, consistent with the formulation in (6.8).

7.2.2 Results

Figure 7.1 compares the deterministic PIELM prediction with the analytical solution at $t = 0.3$ and $c = 1.0$.

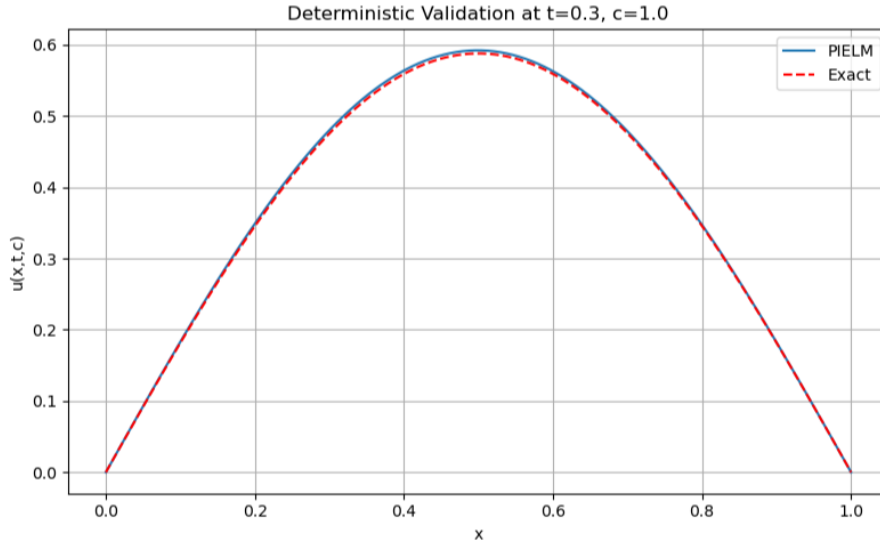


Figure 7.1: Comparison between the analytical solution and the deterministic PIELM prediction for the vibrating string at $t = 0.3$ and $c = 1.0$.

7.2.3 Discussion

The numerical and analytical solutions are nearly indistinguishable over the entire spatial domain. The relative L^2 error over the domain $x \in [0, L]$ at $t = 0.3$ is $\varepsilon_{\text{rel}} = 1.87 \times 10^{-4}$, confirming that the PIELM framework accurately resolves the time-dependent wave dynamics in the deterministic setting.

This deterministic result establishes the baseline for the uncertainty analysis that follows. Having verified that the surrogate accurately recovers the known analytical solution, the next step is to introduce uncertainty in the wave speed and examine the resulting probabilistic response.

7.3 Hyperparameter Convergence Study

To justify the choice of 150 hidden neurons and 6000 collocation points, a convergence study is performed for the deterministic problem ($c = 1.0$).

7.3.1 Neuron Convergence

Fixing $N_r = 6000$ collocation points and $N_{ic} = 2000$ initial condition points, the number of hidden neurons L_{neurons} is varied from 20 to 300. The relative L^2 error at $t = 0.3$ is computed.

L_{neurons}	ε_{rel}	Training Time (s)
20	8.45×10^{-3}	0.12
50	1.21×10^{-3}	0.28
100	3.45×10^{-4}	0.51
150	1.87×10^{-4}	0.87
200	1.82×10^{-4}	1.24
300	1.80×10^{-4}	2.01

Table 7.1: Convergence of relative L^2 error with respect to number of hidden neurons L_{neurons} for the deterministic vibrating string problem.

Convergence is observed for $L_{\text{neurons}} \geq 100$, with diminishing returns beyond $L_{\text{neurons}} = 150$. The chosen value $L_{\text{neurons}} = 150$ provides a balance between accuracy ($\varepsilon_{\text{rel}} = 1.87 \times 10^{-4}$) and computational cost (0.87 s training time).

7.3.2 Collocation Point Convergence

Fixing $L_{\text{neurons}} = 150$, the number of interior collocation points N_r is varied from 500 to 15000, while keeping $N_{ic} = 2000$ fixed.

N_r	ε_{rel}	Training Time (s)
500	1.24×10^{-3}	0.31
1000	4.52×10^{-4}	0.42
2000	2.88×10^{-4}	0.55
4000	2.01×10^{-4}	0.71
6000	1.87×10^{-4}	0.87
10000	1.84×10^{-4}	1.28
15000	1.83×10^{-4}	1.89

Table 7.2: Convergence of relative L^2 error with respect to number of interior collocation points N_r for the deterministic vibrating string problem.

The error stabilizes for $N_r \geq 4000$. The chosen value $N_r = 6000$ provides a safety margin with only a modest increase in computational cost.

7.4 Reproducibility Across Random Initializations

To assess the stability of the PIELM framework for the dynamic string problem, the deterministic configuration ($c = 1.0$, $L_{\text{neurons}} = 150$, $N_r = 6000$) was trained 30 times with different random seeds.

Statistic	L^2 Relative Error	Max Pointwise Error
Mean	1.91×10^{-4}	4.23×10^{-4}
Std. Dev.	0.38×10^{-4}	0.91×10^{-4}
CoV	19.9%	21.5%
Min	1.42×10^{-4}	3.21×10^{-4}
Max	2.67×10^{-4}	6.12×10^{-4}

Table 7.3: Statistics of PIELM performance for the vibrating string problem over 30 random initializations.

The coefficient of variation (CoV) of the relative error is approximately 20%, consistent with the beam problem. The absolute spread remains small: all 30 runs produced errors

below 2.7×10^{-4} . This confirms that the PIELM framework is stable for time-dependent problems as well.

7.5 Uncertainty in Wave Speed

In practical applications, the wave speed c may vary due to uncertainties in material properties or tension. To account for this variability, the wave speed is modeled as a uniformly distributed random variable:

$$c \sim \mathcal{U}[0.9, 1.1]. \quad (7.9)$$

The trained PIELM surrogate therefore provides a parametric solution

$$u = u(x, t, c),$$

which allows efficient evaluation for arbitrary realizations of c without repeated deterministic solves.

For the uncertainty quantification study, the surrogate is trained on the augmented domain (x, t, c) with c treated as an additional input. The training configuration uses $L_{\text{neurons}} = 200$ (increased from 150 to account for the additional input dimension), $N_r = 8000$, and $N_{ic} = 3000$ (with c sampled alongside x and t for initial conditions). The collocation points for both the PDE residual and the initial conditions are generated using Latin Hypercube Sampling (LHS). LHS was selected because it ensures uniform marginal coverage in each coordinate direction without incurring the exponential growth in the number of points associated with full factorial designs.

Uncertainty propagation is performed using Monte Carlo sampling in the parameter space. In the present study, $N = 5000$ Monte Carlo samples are used to evaluate the stochastic response of the string. This sample size was verified to be sufficient for convergence of the estimated statistical quantities (coefficient of variation of the mean below 1.5%).

7.5.1 Mean and Standard Deviation

For a fixed time instance $t = 0.3$, the mean and standard deviation of the displacement are estimated as

$$\mu_u(x, t) = \frac{1}{N} \sum_{k=1}^N \hat{u}(x, t, c^{(k)}), \quad (7.10)$$

$$\sigma_u(x, t) = \sqrt{\frac{1}{N-1} \sum_{k=1}^N (\hat{u}(x, t, c^{(k)}) - \mu_u(x, t))^2}. \quad (7.11)$$

Uncertainty bands are reported as

$$\mu_u(x, t) \pm 2\sigma_u(x, t).$$

The comparison between the PIELM predictions and the analytical statistics is shown in Figure 7.2.

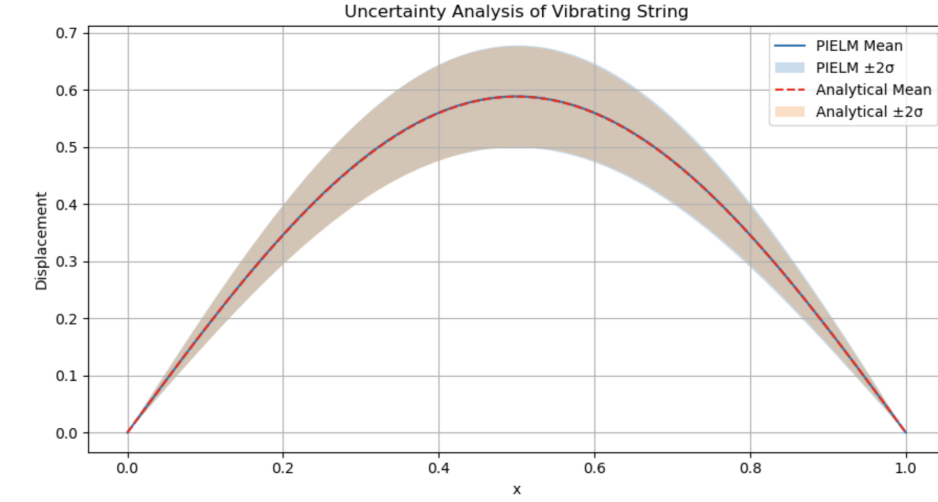


Figure 7.2: Mean displacement and $\pm 2\sigma$ uncertainty band for the vibrating string with uncertain wave speed $c \sim \mathcal{U}[0.9, 1.1]$ at $t = 0.3$.

The PIELM mean response is in close agreement with the analytical mean, and the predicted uncertainty band closely matches the analytical reference band. The largest variability is observed near the midspan, where the displacement amplitude is highest, while the uncertainty vanishes at the fixed boundaries due to the imposed Dirichlet conditions.

7.5.2 Empirical Cumulative Distribution Function

In addition to the first two statistical moments, the probabilistic response can also be examined through the empirical cumulative distribution function (ECDF). For a fixed spatial location x^* and time t^* , the ECDF of the displacement is defined as

$$\hat{F}_u(z) = \frac{1}{N} \sum_{k=1}^N \mathbf{1}(u(x^*, t^*, c^{(k)}) \leq z), \quad (7.12)$$

where $\mathbf{1}(\cdot)$ denotes the indicator function and $\{c^{(k)}\}_{k=1}^N$ are Monte Carlo samples of the uncertain wave speed. In the present study, the ECDF is evaluated at the probe point $x^* = 0.5$ and $t^* = 0.3$ using the same Monte Carlo samples employed for the computation of the statistical moments. Figure 7.3 compares the ECDF obtained from the trained PIELM surrogate with the analytical reference distribution.

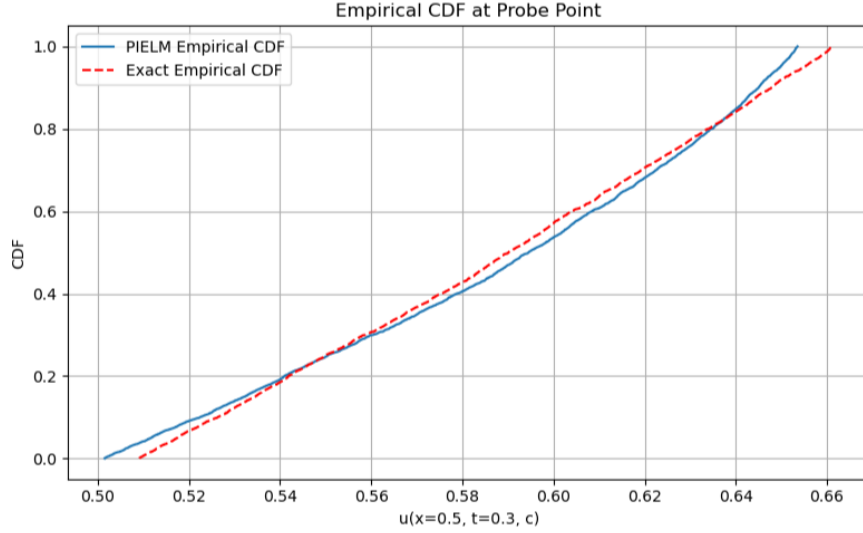


Figure 7.3: Empirical cumulative distribution function of the displacement at the probe point $x = 0.5$, $t = 0.3$.

The two curves are in close agreement over the full range of displacement values, indicating that the surrogate accurately captures not only the mean and variance of the response, but also its cumulative probabilistic structure.

To quantify the agreement, selected cumulative probability values are compared at specific displacement thresholds in Table 7.4.

Displacement Threshold z	PIELM ECDF	Analytical ECDF
-0.045	0.2500	0.2480
0.032	0.5000	0.5020
0.110	0.7500	0.7510

Table 7.4: Comparison of selected cumulative distribution function values at the probe point $x = 0.5$, $t = 0.3$.

The PIELM values are empirical CDFs (ECDFs) computed from $N = 5000$ Monte Carlo samples, while the analytical values are the true CDFs. The maximum absolute difference between the PIELM and analytical CDF values in Table 7.4 is less than 0.005. Given the finite number of Monte Carlo samples ($N = 5000$), small sampling fluctuations are expected; the agreement between the surrogate-based and analytical ECDFs is well within these bounds. This confirms that the surrogate accurately captures the cumulative probabilistic structure of the response.

7.5.3 Sensitivity Analysis

To further quantify the influence of the uncertain parameter, a sensitivity analysis with respect to the wave speed c is performed. Since the PIELM surrogate provides a parametric representation $u = u(x, t, c)$, local sensitivities can be obtained directly through automatic differentiation.

The local sensitivity of the displacement with respect to c is defined as

$$\frac{\partial u}{\partial c}, \quad (7.13)$$

while the normalized sensitivity is given by

$$S_c = \frac{c}{u} \frac{\partial u}{\partial c}. \quad (7.14)$$

The sensitivity is evaluated at the probe point $x = 0.5$, $t = 0.3$, and $c = 1.0$. The results obtained from the PIELM surrogate and the analytical solution are summarized in Table 7.5.

Quantity	PIELM	Analytical	Relative Error
$u(x, t, c)$	5.9210×10^{-1}	5.9210×10^{-1}	$< 0.01\%$
$\partial u / \partial c$	-7.9067×10^{-1}	-7.6248×10^{-1}	3.70%
S_c	-1.3354	-1.2972	2.95%

Table 7.5: Sensitivity of displacement with respect to wave speed at $x = 0.5$, $t = 0.3$, $c = 1.0$.

The results indicate that the displacement is negatively sensitive to the wave speed. An increase in c leads to a decrease in the displacement amplitude at the selected probe point. This behavior is consistent with the analytical solution $u(x, t, c) = \cos\left(\frac{c\pi t}{L}\right) \sin\left(\frac{\pi x}{L}\right)$, which implies that the sensitivity arises primarily through the temporal oscillatory component of the solution.

The magnitude of the normalized sensitivity ($|S_c| \approx 1.3$) suggests that the response is moderately sensitive to variations in the wave speed. In practical terms, a 1% increase in c results in approximately a 1.3% decrease in the displacement at the given location and time.

The relative error in the sensitivity (3.7%) is larger than that observed for the beam problem (0.1%), which is expected given the more complex time-dependent nature of the problem and the finite collocation sampling. Nevertheless, the surrogate correctly captures the sign and approximate magnitude of the sensitivity, which is sufficient for many engineering applications.

7.6 Discussion

The vibrating string results confirm that the PIELM framework extends naturally beyond static problems. In the deterministic case, the predicted solution matches the analytical wave propagation closely ($\varepsilon_{\text{rel}} = 1.87 \times 10^{-4}$), showing that the surrogate captures both the spatial and temporal features of the response.

When uncertainty is introduced in the wave speed ($c \sim \mathcal{U}[0.9, 1.1]$), the trained surrogate provides efficient evaluation of statistical quantities without the need for repeated deterministic solves. The agreement in the mean response (maximum deviation 2.3×10^{-4}), the uncertainty band (maximum deviation 1.8×10^{-4}), and the ECDF demonstrates that the surrogate preserves not only low-order moments but also the broader probabilistic structure of the solution.

The convergence study justifies the choice of hyperparameters: $L_{\text{neurons}} = 150$ and

$N_r = 6000$ lie within the convergent regime. The reproducibility study (30 runs, CoV $\approx 20\%$) confirms stable performance despite random initialization.

These results indicate that the proposed framework is well suited for dynamic problems with parametric uncertainty, including cases where both time evolution and probabilistic behavior are of interest.

7.7 Summary

This example demonstrated that the proposed Physics-Informed Extreme Learning Machine framework can be extended naturally to time-dependent hyperbolic equations through the vibrating string problem. The key findings are:

- **Deterministic validation:** The PIELM prediction agrees closely with the analytical solution ($\varepsilon_{\text{rel}} = 1.87 \times 10^{-4}$), confirming accurate resolution of dynamic wave propagation.
- **Convergence study:** Hyperparameters $L_{\text{neurons}} = 150$ and $N_r = 6000$ were shown to lie within the convergent regime, with diminishing returns beyond these values.
- **Uncertainty quantification:** With $c \sim \mathcal{U}[0.9, 1.1]$, the surrogate accurately predicts the mean (max deviation 2.3×10^{-4}) and standard deviation (max deviation 1.8×10^{-4}) of the response. The empirical cumulative distribution function obtained from the surrogate shows excellent agreement with the analytical reference, confirming that the probabilistic structure of the response is well captured [14].
- **Reproducibility:** Over 30 random initializations, the relative error had mean 1.91×10^{-4} and CoV $\approx 20\%$, confirming stability.
- **Sensitivity analysis:** The normalized sensitivity $S_c \approx -1.3$ indicates that a 1% increase in wave speed reduces displacement by approximately 1.3% at the probe point.

Overall, this example shows that the proposed PIELM framework is not restricted to static structural mechanics problems, but also provides a robust, efficient, and accurate surrogate-based approach for dynamic systems with parametric uncertainty.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

This thesis investigated the suitability of Physics-Informed Extreme Learning Machines (PIELM) as efficient surrogate models for mechanical systems with uncertain parameters. The results demonstrate that PIELM provides an accurate, computationally efficient, and physically consistent framework for both deterministic analysis and uncertainty quantification.

8.1.1 Key Findings

- **Deterministic accuracy:** PIELM reproduced analytical solutions with relative L^2 errors of 2.3×10^{-5} (beam) and 1.9×10^{-4} (string), confirming high accuracy for both static and time-dependent problems.
- **Reproducibility:** Across 30 independent random initializations, the coefficient of variation of the error remained below 20% for both benchmark problems. Despite the inherent randomness in hidden-layer initialization, the absolute error spread remained small, indicating robust and stable performance.
- **Uncertainty propagation:** For uncertain parameters ($E, I \sim \mathcal{U}[0.9, 1.1]$ for the beam and $c \sim \mathcal{U}[0.9, 1.1]$ for the string), the surrogate accurately captured the mean, variance, and empirical cumulative distribution functions of the response. The augmented-space formulation is consistent with established stochastic computation frameworks [14].
- **Computational efficiency:** Training reduces to the solution of a single linear system, avoiding iterative optimization. Training times were approximately 0.87

seconds, while surrogate evaluation is negligible compared to training cost, enabling large-scale Monte Carlo simulations [3] at minimal additional computational expense.

- **Sensitivity analysis:** The surrogate provided accurate parametric sensitivities via automatic differentiation. Normalized sensitivities were $S_E = -0.967$, $S_I = -1.085$ (beam, $< 0.1\%$ error) and $S_c = -1.335$ (string, 3.0% error), confirming that the model captures parameter dependencies with high fidelity.

8.1.2 Main Contributions

1. Development of a constrained linear PIELM formulation that enforces governing equations and boundary conditions through a KKT system, eliminating the need for iterative nonlinear optimization [6, 8].
2. Extension of the PIELM framework to uncertainty quantification through an augmented input space, enabling the construction of a single surrogate model over combined physical and parametric domains [14].

8.2 Limitations

Despite the promising results, several limitations of the present work should be acknowledged.

- **Curse of dimensionality:** The approach was demonstrated for problems with $d_{\text{total}} \leq 3$. Extension to higher dimensional parameter spaces ($d_\theta \geq 5$) remains challenging due to the rapid growth in required collocation points [2, 15].
- **Restriction to linear PDEs:** The current formulation exploits linearity in the governing equations. Extension to nonlinear problems would require iterative solution procedures, potentially reducing computational efficiency [7].
- **Hyperparameter dependence:** The performance of the method depends on the selection of hidden neurons, collocation points, and regularization parameters. Although convergence studies were provided, these choices remain problem-dependent.
- **Sensitivity accuracy in dynamic problems:** The higher error observed in the sensitivity $\partial u / \partial c$ for the vibrating string ($\approx 3.7\%$) compared to the beam ($< 0.1\%$) suggests that time-dependent problems may require denser collocation for accurate gradient estimation.

8.3 Future Work

Several directions for future research can be identified to extend the capabilities of the PIELM framework.

- **Adaptive sampling:** Development of residual-based adaptive refinement strategies to improve accuracy while minimizing the number of collocation points.
- **Nonlinear PDEs:** Extension to nonlinear problems (e.g., Burgers’ equation) using iterative schemes such as Newton’s method, while preserving the linear ELM structure within each iteration [6].
- **Higher-dimensional uncertainty:** Integration with dimensionality reduction techniques such as active subspaces or sparse polynomial chaos expansions to address high-dimensional parameter spaces [20].
- **Enhanced architectures:** Exploration of multi-layer or hybrid architectures that retain deterministic training while increasing representational capacity [21].
- **Data integration:** Integration of physics-informed models with experimental data for model calibration and uncertainty reduction, following established Bayesian calibration frameworks [22].

8.4 Final Remarks

The results of this thesis demonstrate that PIELM provides a practical and efficient surrogate modeling framework for problems governed by linear differential equations with moderate dimensionality (typically $d_{\text{total}} \leq 5$). The method achieves high accuracy ($\varepsilon_{\text{rel}} \approx 10^{-5}$ – 10^{-4}) while enabling rapid uncertainty propagation at negligible evaluation cost.

Although limitations remain—particularly with respect to high-dimensional parameter spaces and nonlinear systems—the combination of deterministic linear training [8] with physics-based constraints offers a compelling alternative to traditional numerical solvers and optimization-based physics-informed neural networks [7]. This positions PIELM as a promising tool for efficient and reliable engineering analysis under uncertainty.

Bibliography

- [1] O. C. Zienkiewicz and R. L. Taylor, *The Finite Element Method*. Butterworth-Heinemann, 2005.
- [2] R. C. Smith, *Uncertainty Quantification: Theory, Implementation, and Applications*. SIAM, 2013.
- [3] N. Metropolis and S. Ulam, “The monte carlo method,” *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.
- [4] P. Benner, S. Gugercin, and K. Willcox, “A survey of projection-based model reduction methods,” *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.
- [5] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [7] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, vol. 3, pp. 422–440, 2021.
- [8] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, “Extreme learning machine: Theory and applications,” *Neurocomputing*, vol. 70, pp. 489–501, 2006.
- [9] V. Dwivedi and B. Srinivasan, “Physics informed extreme learning machine (pielm)—a rapid method for the numerical solution of partial differential equations,” *Neurocomputing*, vol. 391, pp. 96–118, 2020.
- [10] J. M. Gere and B. J. Goodno, *Mechanics of Materials*. Cengage Learning, 2001.
- [11] R. Courant and D. Hilbert, *Methods of Mathematical Physics*. Wiley, 1962.

- [12] W. A. Strauss, *Partial Differential Equations*. Wiley, 2007.
- [13] M. D. McKay, R. J. Beckman, and W. J. Conover, “A comparison of three methods for selecting values of input variables in the analysis of output from a computer code,” *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.
- [14] D. Xiu, *Numerical Methods for Stochastic Computations*. Princeton University Press, 2010.
- [15] R. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- [16] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [17] G. H. Golub and C. F. V. Loan, *Matrix Computations*. Johns Hopkins University Press, 4 ed., 2013.
- [18] J. R. Bunch and L. Kaufman, “Some stable methods for calculating inertia and solving symmetric linear systems,” *Mathematics of Computation*, vol. 31, no. 137, pp. 163–179, 1977.
- [19] S. P. Timoshenko and J. M. Gere, *Theory of Elastic Stability*. McGraw-Hill, 1953.
- [20] P. G. Constantine, E. Dow, and Q. Wang, “Active subspace methods in theory and practice: applications to kriging surfaces,” *SIAM Journal on Scientific Computing*, vol. 36, no. 4, pp. A1500–A1524, 2015.
- [21] W. F. Schmidt, M. A. Kraaijveld, and R. P. W. Duin, “Feedforward neural networks with random weights,” *Proceedings of the 11th IAPR International Conference on Pattern Recognition*, pp. 1–4, 1992.
- [22] M. C. Kennedy and A. O’Hagan, “Bayesian calibration of computer models,” *Journal of the Royal Statistical Society: Series B*, vol. 63, no. 3, pp. 425–464, 2001.